CSE3001
Database Management Systems

Prof. Rajneesh Kumar Patel

Expected Outcomes:

- Advantages of using a DBMS rather than a file system.
- Designing an Entity-Relationship model for a real-life application.
- Mapping a database schema from ER model.
- Evaluating relational schemas for design qualities
- Optimize a query.
- Basic concepts on transaction processing, concurrency control and recovery.
- Fundamental view on unstructured data and its management.
- Storage of databases and techniques to access them using various algorithms

Expected Outcomes:

- Explain the role of a database management system in an organization.
- Describe the structure and operation of the relational data model.
- Construct database queries using Structured Query Language (SQL).
- Design and implement a database project depending on the business requirements, considering various design issues.
- Implement the concept of a database transaction and related database facilities, including concurrency control, backup and recovery.

Modules	Syllabus
Module-1	Introduction to Database system: purpose of database system, view of data, Independence, relational databases, -Database Language-Database System architecture- levels, Mappings, Database, users and DBA Data Models: Importance, Basic building blocks, Degrees of data abstraction. Database design and ER Model: Overview, ER-Model, Constraints, ER Diagrams, ERD Issues, weak entity sets.
Module-2	Relational Models: Structure of relational databases, Domains, Relations, Relational algebra – fundamental operators and syntax, selection, Projection, relational algebra queries, tuple relational calculus, set operations, renaming, Joins, Division, syntax. Operators, grouping and ungrouping, relational comparison. Codd's rules, Relational Schemas, Introduction to UML Relational database model: Logical view of data, keys, integrity rules. Normalization: 1NF, 2NF, 3NF, BCNF, Multi valued dependencies and Fourth Normal Form.
Module-3	Relational Database design: features, atomic domain. SQL: data definition, aggregate function, Null Values, nested sub queries, Joined relations. Triggers. Data Manipulation Language (DML) and Transaction Control Language (TCL) , Basic SELECT statements , Table Joins, Restricting and Sorting Data - Order By / Group By. Retrieving Data Using the SQL SELECT Statement, Single-Row Functions, Conversion Functions and Conditional Expressions, Reporting Aggregated Data Using the Group Functions, Displaying Data from Multiple Tables. Joins, Set Operators, DML Statements, Data Definition Language - Data Dictionary Views - Creating Sequences, Synonyms, and Indexes, Creating Views - Introduction to views, data independence, security, updates on views, comparison between tables and views.

Modules	Syllabus
Module-4	PL/SQL: Declaring PL/SQL Variables, Writing Executable Statements, Using SQL Statements Within a PL/SQL Block, Control Structures, Composite Data Types, Using Explicit Cursors, Handling Exceptions, Introducing Stored Procedures and Functions. Data Storage: Overview of Physical Storage Media - Magnetic disk Flash storage - RAID-File and Record Organization-Indexing and Hashing, Ordered Indices - B+ Tree Index File-Static Hashing -Dynamic Hashing Query Processing: Overview-measures of Query Cost. Query Optimization Techniques- Cost based Optimization-Heuristic Optimization
Module-5	Transaction Management: Transaction concepts-properties of transactions-Transaction Atomicity and Durability-Transaction Isolation- Implementation of Isolation Levels-serializability of transactions, testing for serializability. Concurrency Control: Lock based protocols--Deadlock handling-Multiple Granularity-Time stamp based protocols -System recovery, Two- Phase Commit protocol, Recovery system: Failure classification --Recovery and atomicity Recovery Algorithms.

Introduction to Database:

- **What is Data?**
  - Collection of raw facts.
  - Facts that can be recorded and have implicit meaning, known as data.
  - For ex: Students records- rno, name, address…

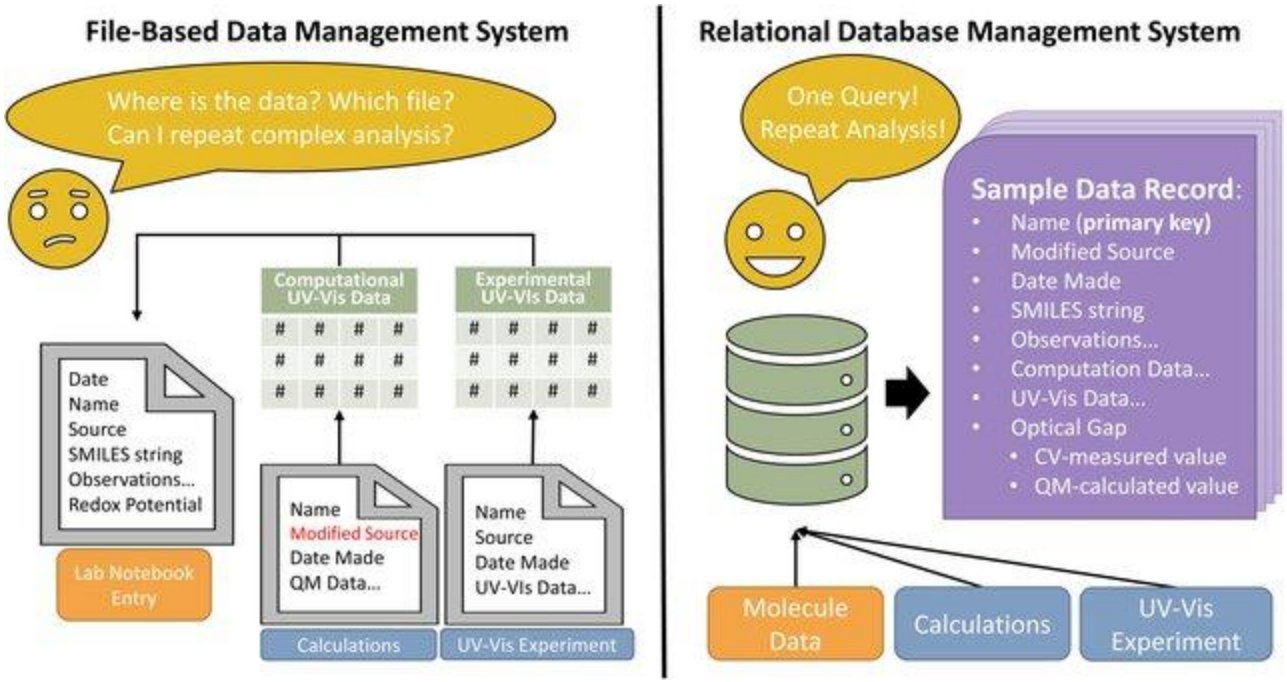
- **Database**
  - Collection of interrelated data.
  - Can be stored in the form of tables.
  - Data can be stored, updated, deleted either manually or computerized.

Rno	Name	Address
101	Ajay	Bhopal
102	Sahil	Ujjain

Introduction to Database Cont….:

- **Database Management System**
  - DBMS is collection of interrelated data items and set of programs to easily store, update and retrieve those data.
  - **Interrelated Data + Set of programs Instructions**
  - Primary goal of DBMS is to perform the operations for data without any anomalies.

Database System vs Traditional File System



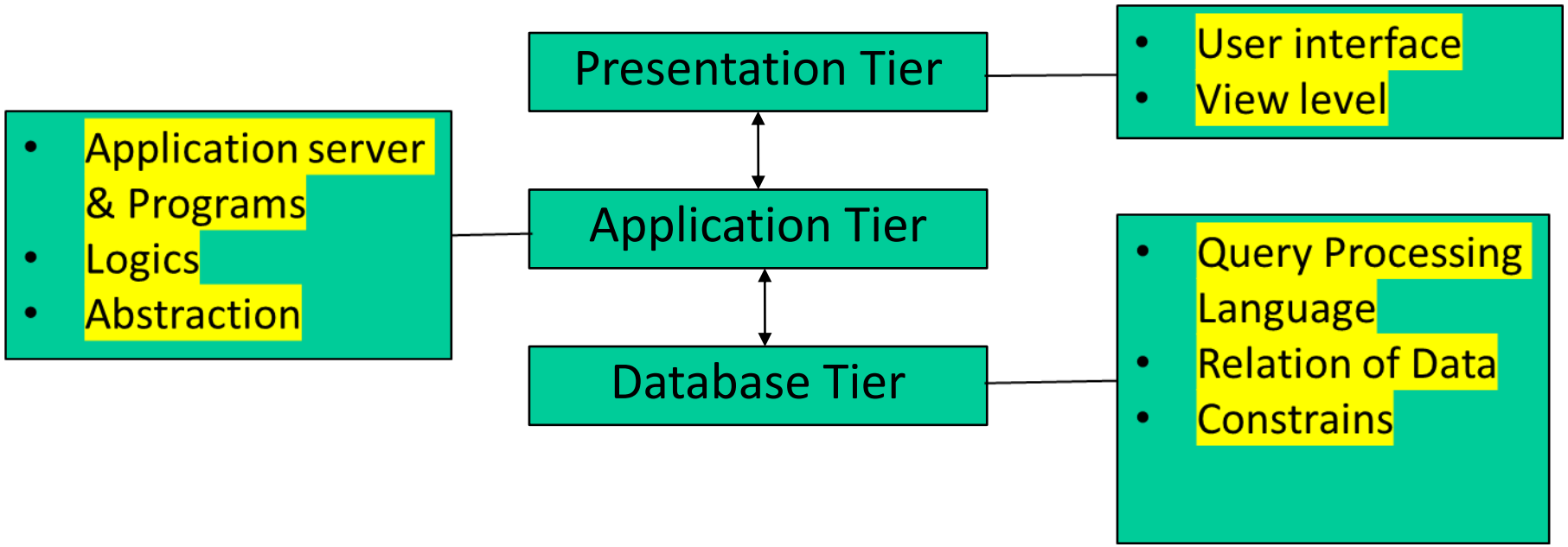
Database System vs Traditional File System

- **Disadvantages of File Processing Systems**
  - Data Redundancy
  - Difficulties in accessing the data
  - Data Isolation
  - Atomicity (All or None)
  - Concurrency Control Anomalies
  - Security
  - Integrity Constrains
    - Domain Constrains
    - Key Constrains
    - Referential Integrity

File-Base Approach	Database Approach
1. The program and data are interdependent.	1. The program and data are independent of each other.
2. The data may be duplicated in different files that cause data redundancy.	2. The data is not duplicated and appears only once.
3. The same data in different files may be different that creates inconsistency.	3. Data appears only once so it is always consistent.
4. The data is separately stored in different files and it is difficult to application to retrieve the appropriate data.	4. The data is stored in tables which are linked together. The application can retrieve the required data easily.
5. The data is distributed in many different files and cannot be shared.	5. The data is stored at one place and can be shared easily.
6. It is difficult to apply data integrity checks on files.	6. Database approach provides many constraints for data integrity.
7. It provides poor security as the data is widely spread.	7. It provides many procedures to maintain data security.
8. It is difficult to maintain as it provides less controlling facilities.	8. It provides many facilities to maintain the programs easily.
9. It is less complex system.	9. It is very complex system.
10. The cost is very less than DBMS.	10. The cost is much more than file system.
11. If one application fails it did not affect the other.	11. All application relying on database fail if the database fails.

DBMS Architecture:

- Three Tier Architecture of DBMS:



DBMS Architecture

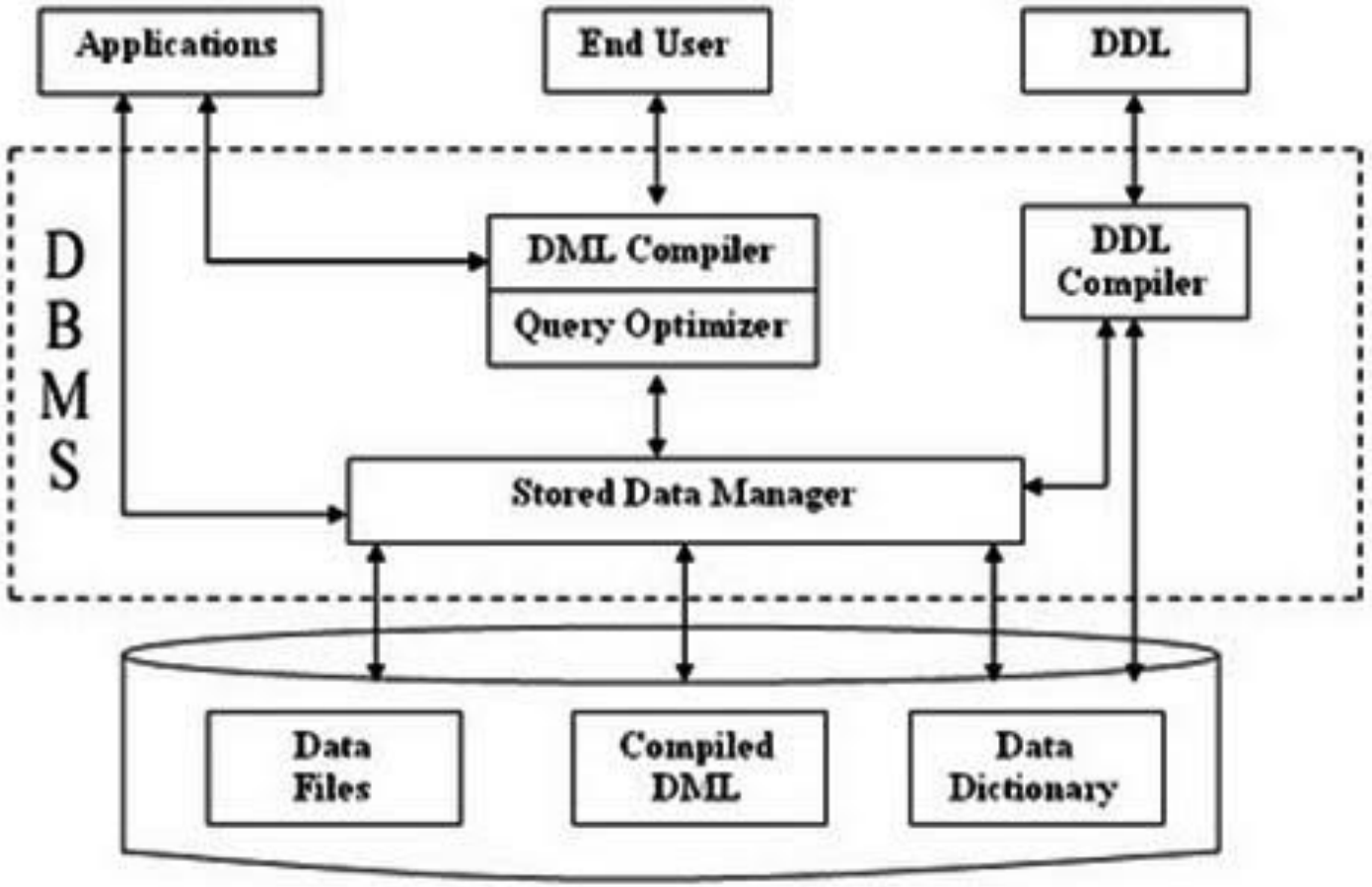
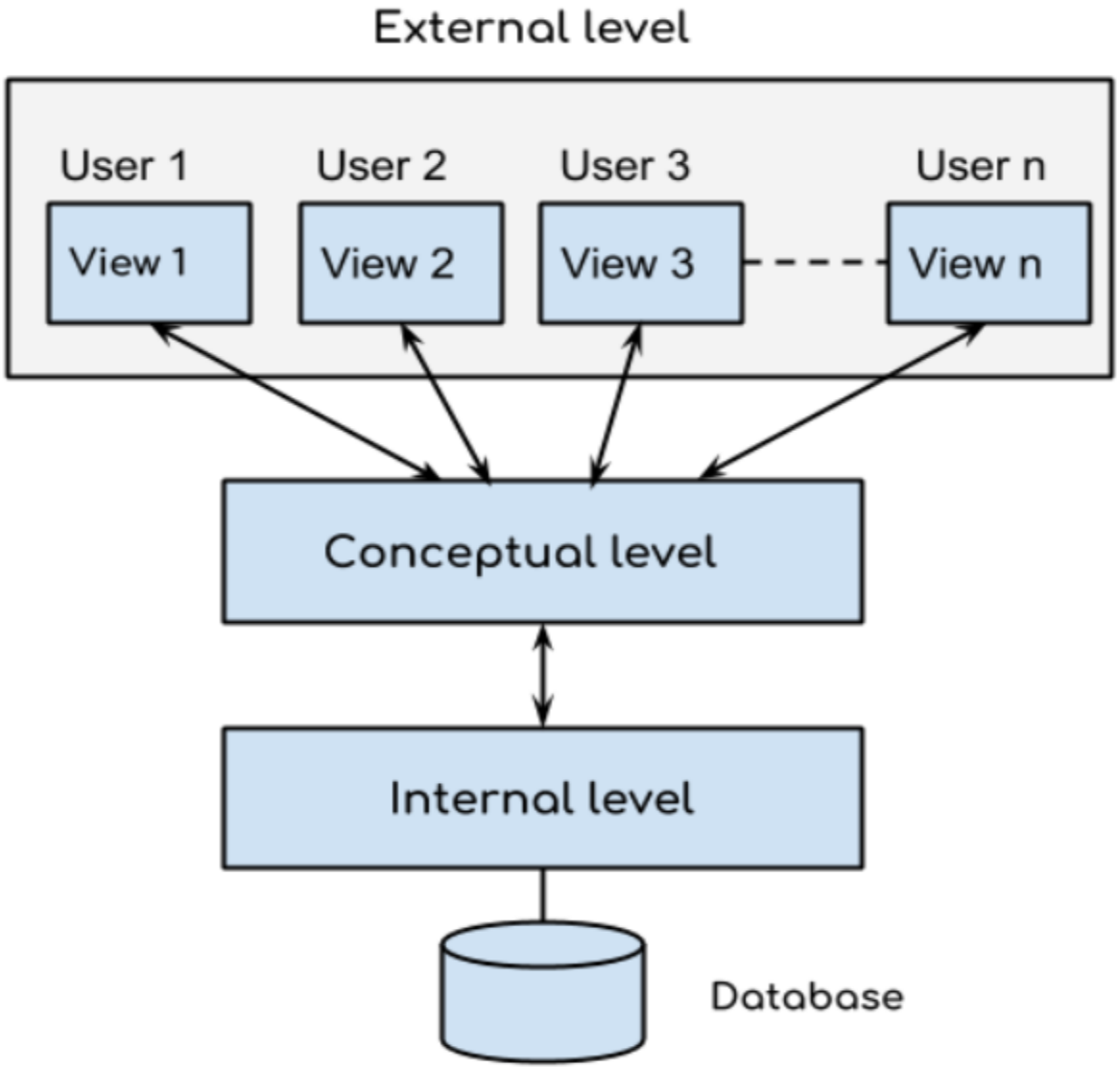


Fig. 2.1 Structure of DBMS

Three Level Database Schema Architecture:



Three Level Schema Architecture:

1. Internal Level

- The internal level schema describes the physical storage and data structure of the database.
- It is also known as a physical schema and it uses physical data model.
- It is used to define that how the data will be stored in a block.

2. Conceptual Level

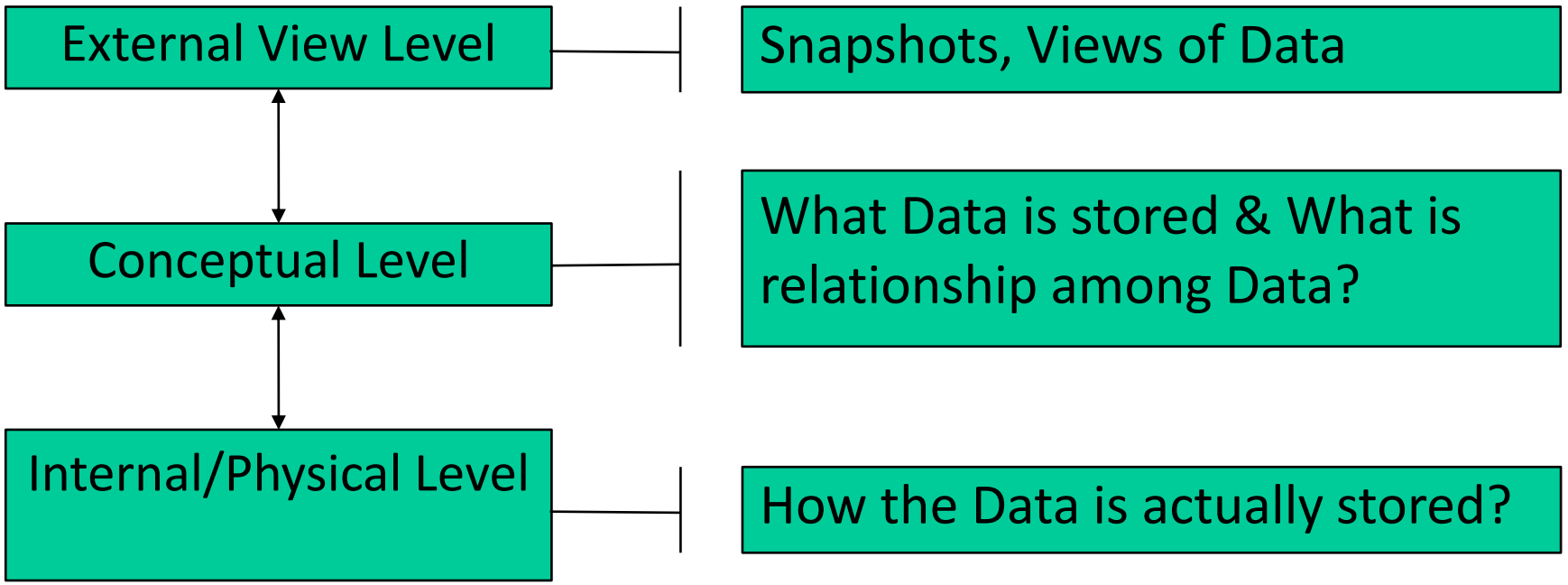
- The conceptual schema describes the design of a database at the conceptual level.
- Conceptual level is also known as logical level.
- The conceptual level describes what data are to be stored in the database and also describes what relationship exists among those data.
- In this level, internal details such as an implementation of the data structure are hidden.
- Programmers and database administrators work at this level.

3. External Level

- The external level describes the different views of the database.
- An external schema is also known as view schema.
- Each view schema describes the database part that a particular user group is interested and hides the remaining database from that user group.
- The view schema describes the end user interaction with database systems.

Levels of Data Abstraction or View of Data:

- **Three levels of Data Abstraction**



Data Independence:

- A Database System that is able to separate three different levels of abstraction is likely to be flexible and adaptable.
- This flexibility and adaptability is called DATA INDEPENDENCE.
- Means if we change in low level then higher level should not be affected.

- **Two types of Data Independence:**
  - Physical Data Independence:
    - Separation of Conceptual & View from Internal View
    - Without specifying the physical structure of Database
  - Logical Data Independence
    - Separation of external view from conceptual view
    - Changes in concepts without affecting external view

Database Schemas & Instances:

Database Schemas:

- It is the description of Database (Overall design of Database).
- Specified during database design.
- Not expected to change frequently.

Student (Roll_No Varchar(10), Name Char(30), Address varchar(30), Phone_No Number(10), DOB date……..)

Department (Dept_Id Varchar(10), Dept_Name Char(10), Course_Name Char(20)…….)

Database Schemas & Instances Cont.…:

Database Instances:

- Collection of information stored in the database at particular state of time.
- Snapshot of Database.

- **Find those students who lives in Bhopal-**

Roll No	Name	Contact No	Address
101	Ajay	123	BPL
125	Kartik	456	BPL
140	Rahul	789	BPL

Database Languages:

1. DDL (Data Definition Language):

- It is used to define database structure or pattern.
- It is used to create schemas, tables, indexes, constraints, etc. in the database.
- Data definition language is used to store the information of metadata like the number of tables and schemas, their names, indexes, columns in each table, constraints, etc.

Here are some Commands that come under DDL:

Create: It is used to create objects in the database.

Alter: It is used to alter the structure of the database.

Drop: It is used to delete objects from the database.

Truncate: It is used to remove all records from a table.

Rename: It is used to rename an object.

Data Definition Language:

SQL> create table student_detail(roll_no number(10), name char(30));
Table created.

SQL> alter table student_detail add(contact_no number(10), address varchar(20));
Table altered.

SQL> alter table student_detail modify(roll_no varchar(20) primary key);
Table altered.

SQL> alter table student_detail drop(contact_no);
Table altered.

SQL> rename student_detail to student;
Table renamed.

Data Definition Language:

SQL> desc student;

Name	Null?	Type
ROLL_NO	NOT NULL	VARCHAR2(20)
NAME		CHAR(30)
ADDRESS		VARCHAR2(20)

SQL> truncate table student;

Table truncated.

SQL> drop table student;

Table dropped.

Database Languages Cont.…:

2. DML (Data Manipulation Language):

- It is used for accessing and manipulating data in a database.
- It handles user requests.

Here are some Commands that come under DML:

Select: It is used to retrieve data from a database.

Insert: It is used to insert data into a table.

Update: It is used to update existing data within a table.

Delete: It is used to delete all records from a table.

Data Manipulation Language:

```
SQL> insert into student values ('CS101', 'Ajay', 'Bhopal');

SQL> insert into student values ('CS102', 'Aman', 'Ujjain');

SQL> insert into student values ('CS103', 'Amit', '');

SQL> insert into student (roll_no) values ('CS104');

SQL> insert into student (name) values ('Anuj');
insert into student (name) values ('Anuj')
*
ERROR at line 1:
ORA-01400: cannot insert NULL into ("SYSTEM"."STUDENT"."ROLL_NO")

SQL> insert into student (roll_no, name) values ('CS105','Anuj');
```

Data Manipulation Language:

SQL> select *from student;

ROLL_NO	NAME	ADDRESS
CS101	Ajay	Bhopal
CS102	Aman	Ujjain
CS103	Amit	
CS104		
CS105	Anuj	

SQL> select roll_no, name from student;

Data Manipulation Language:

SQL> select * from student where address= 'Bhopal';

ROLL_NO	NAME	ADDRESS
CS101	Ajay	Bhopal

SQL> select roll_no as roll_number from student;

ROLL_NUMBER
CS101
CS102
CS103
CS104
CS105

Data Manipulation Language:

SQL> update student set address='Bhopal' where roll_no='CS103';

SQL> update student set name='Anay', address='Ujjain' where roll_no='CS104';

SQL> update student set address='Indore' where name='Anuj';

SQL> select * from student;

ROLL_NO	NAME	ADDRESS
CS101	Ajay	Bhopal
CS102	Aman	Ujjain
CS103	Amit	Bhopal
CS104	Anay	Ujjain
CS105	Anuj	Indore

Data Manipulation Language:

SQL> delete from student where roll_no='CS104';

1 row deleted.

SQL> delete from student where address='Bhopal';

2 rows deleted.

SQL> select * from student;

ROLL_NO	NAME	ADDRESS
CS102	Aman	Ujjain
CS105	Anuj	Indore

Database Languages Cont.…:

3. DCL (Data Control Language):

- It is used to retrieve the stored or saved data.

Here are some Commands that come under DCL:

Grant: It is used to give user access privileges to a database.

Revoke: It is used to take back permissions from the user.

4. TCL (Transaction Control Language):

- TCL is used to run the changes made by the DML statement.
- TCL can be grouped into a logical transaction.

Here are some Commands that come under DCL:

Commit: It is used to save the transaction on the database.

Rollback: It is used to restore the database to original since the last Commit.

Functions of DBA:

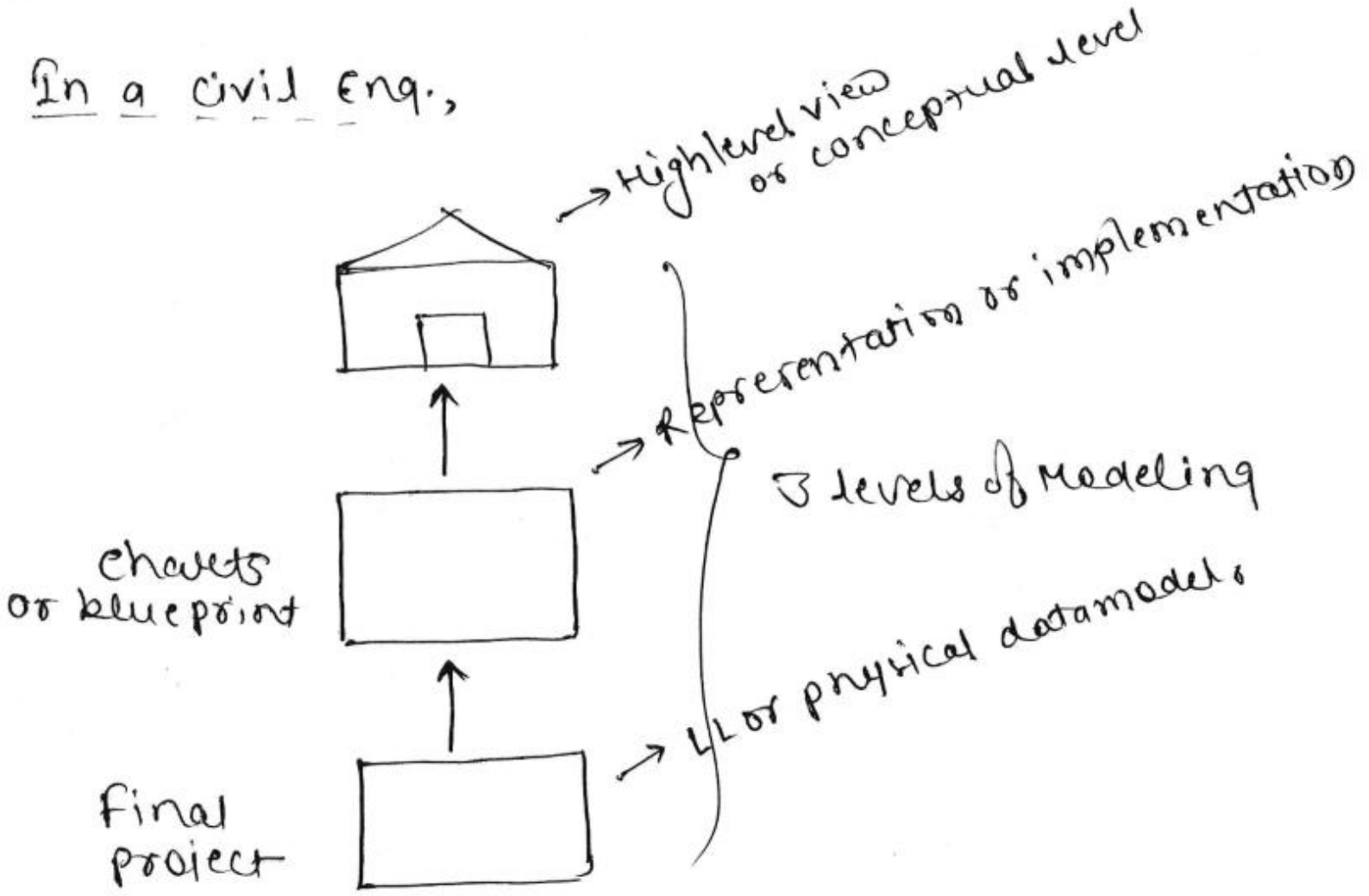
1. Designing Conceptual & Physical Schema
2. Security & Authorization
3. Data Availability & Recovery from failures
4. Storage Structure and Access Method Definitions
5. Integrity Constrains Specifications

Data Models:

- Collection of concept that can be used to describe the structure of DBMS like datatypes, relationships and constrains.
- Specifies the dynamic aspects and behavior of database applications.
- Relatively simple representations, usually graphical of complex real-world data structures.
- Facilitate interaction among the designer, applications programmer and the end user.

Database Models :-

eg. In a civil Eng.,



In dbms,



High level or conceptual view → ER Model



Representation or implementation → Relational Model → Tables
(for programmers → SQL, Tables)



Low level or Physical data models → How tables are stored in mem
structure of records, data types, data retrieval.

Evolution of Data Models:

- **High Level or Object Based Data Model**
  - Entity Relationship Model
  - Object Oriented Data Model

- **Representation or Record Based Data Model**
  - Relational Model
  - Network Model
  - Hierarchical Model

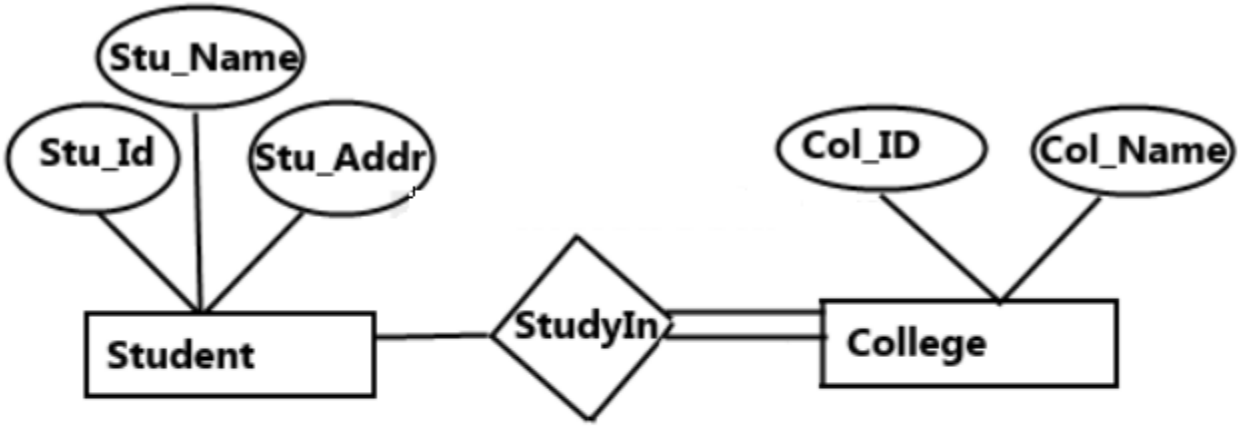
- **Low Level or Physical Data Model**
  - Unifying Model
  - Frame Memory Model

Entity Relationship Model

- Widely accepted and adapted graphical tool for data modeling.
- ER Model is a high-level conceptual data model diagram.
- ER model helps you to analyze data requirements to produce a well-designed database.
- Graphical representation of entities and their relationships in a database structure.
- In other words, ER diagrams help to explain the logical structure of databases.

- **Components of ER Model:**
  - **Entity**
  - **Relationship**
  - **Attributes**

Entity Relationship Model Example-



E-R Digram with total participation of College entity set in StudyIn relationship Set - This indicates that each college must have atleast one associated Student.

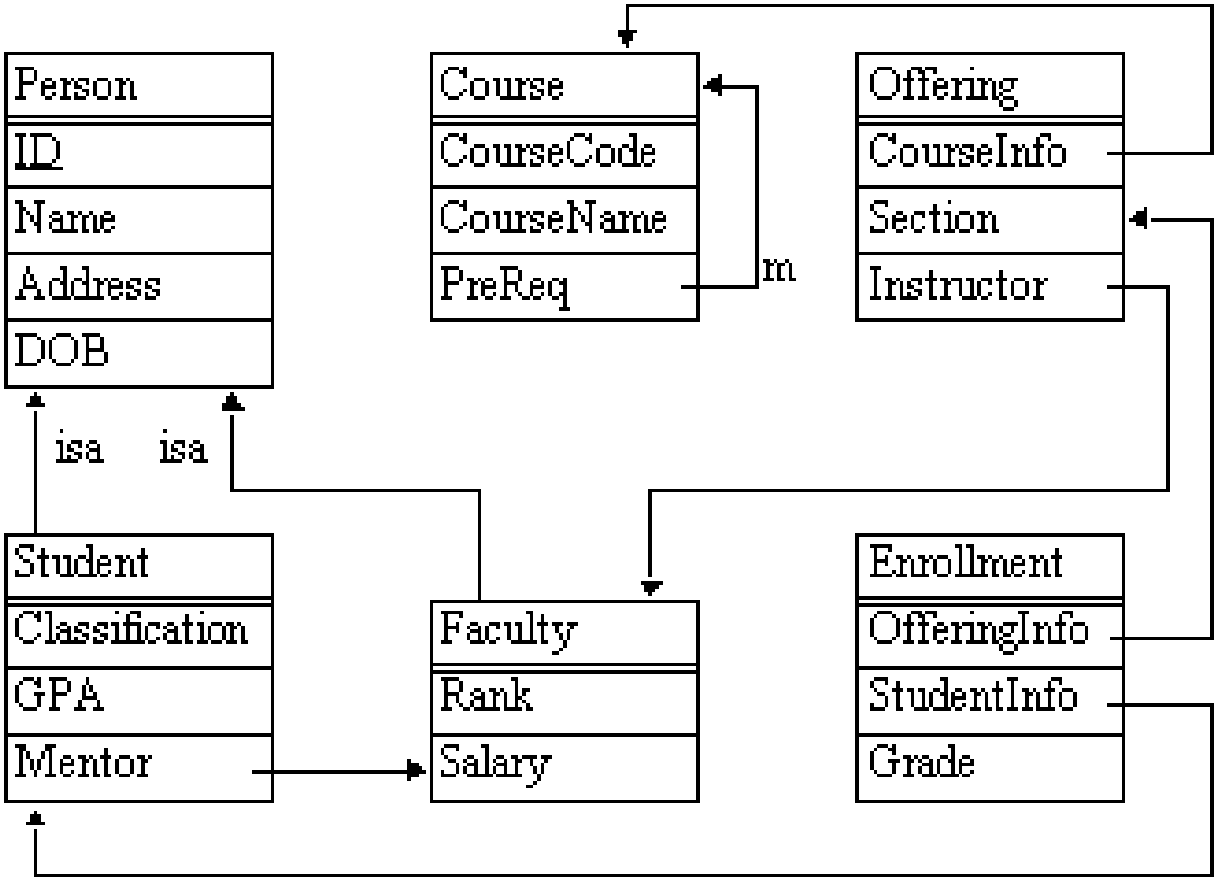
Object-Oriented Model

- Modeled both data and their relationships in a single structure known as an object.
- Object-oriented data model (OODM) is the basis for the object-oriented database management system (OODBMS).
- Object becomes basic building block for autonomous structures.

Components of OODM: Object and Class

- Object is an abstraction of a real-world entity.
- Attributes describe the properties of an object.
- Objects that share similar characteristics are grouped in classes.
- Inheritance is the ability of an object within the class hierarchy to inherit the attributes and methods of classes above it.

Object-Oriented Model Example-



Relational Model

- Developed by Codd (IBM) in 1970.
- Conceptually simple.
- Extension for Relational Database Management System (RDBMS).
- Most important advantage of the RDBMS is its ability to hide the complexities of the relational model from the user.

A Relational Model-

- Representation of relational database's entities, attributes within those entities, and relationships between those entities.

The Relational Model Cont.....

Relational Table (relations)

- Stores a collection of related entities
- Matrix consisting of a series of row/column intersections
- Relational table is purely logical structure
- How data are physically stored in the database is of no concern to the user or the designer.
- This property became the source of a real database revolution.

Structured Query Language (SQL)

- Allows users to specify what must be done without specifying how it must be done.
- SQL-based relational database application involves- User interface, set of tables stored in the database and SQL engine.

The Relational Model Example-

itemid	orderid	item	amount
5	1	Chair	200.00
6	1	Table	200.00
7	1	Lamp	123.12

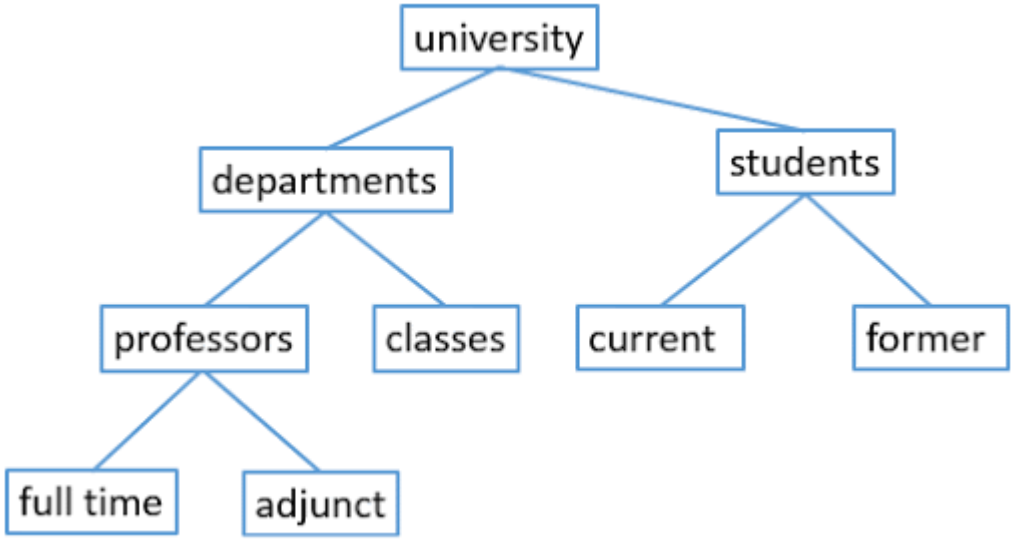
customerid	name	email
5	Rosalyn Rivera	rosalyn@adatum.com
6	Jayne Sargent	jayne@contoso.com
7	Dean Luong	dean@contoso.com

orderid	customerid	date	amount
1	4	11/1/17	523.12
2	3	11/15/17	32.99
3	1	11/21/17	23.99

The Hierarchical Model

- Developed in the 1960s to manage large amounts of data for complex manufacturing projects.
- Basic logical structure is represented by an upside-down "tree"
- The hierarchical structure contains levels or segments.
- Depicts a set of one-to-many (1:M) relationships between a parent and children's segments.

The Hierarchical Model Example-



The Hierarchical Model Cont.....

Advantages

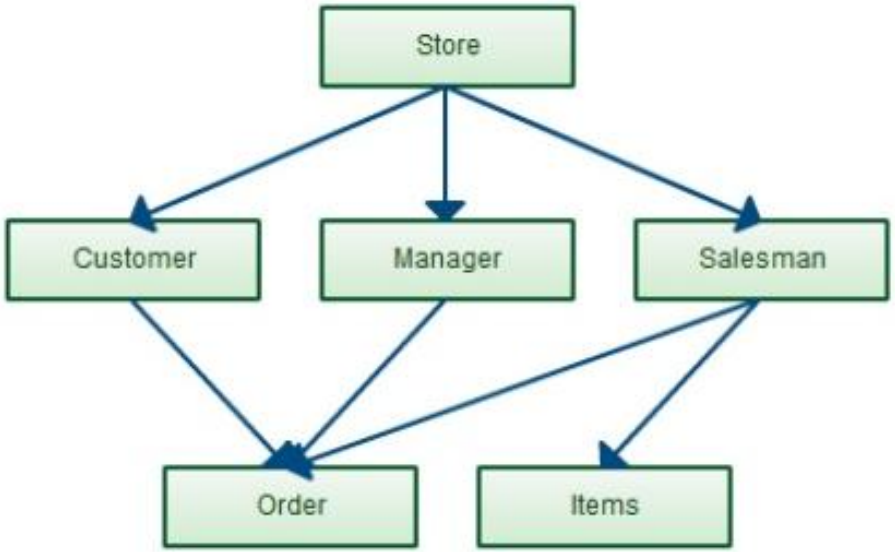
- Many of the hierarchical data model's features formed the foundation for current data models.
- Its database application advantages are replicated in current database environments.

Disadvantages

- Complex to implement
- Difficult to manage
- Implementation limitations

The Network Model

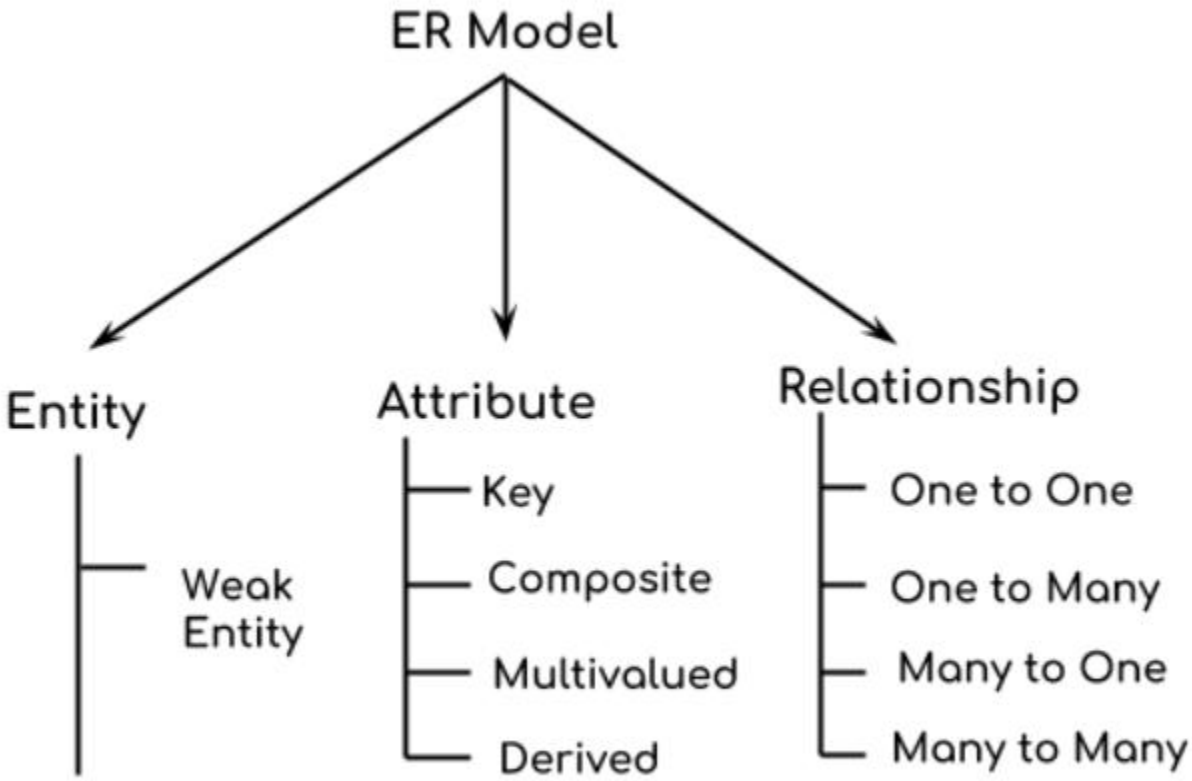
- Developed to represent complex data relationships more effectively.
- Improve database performance with the help of links/ connections.
- Collection of records in M:M relationships.



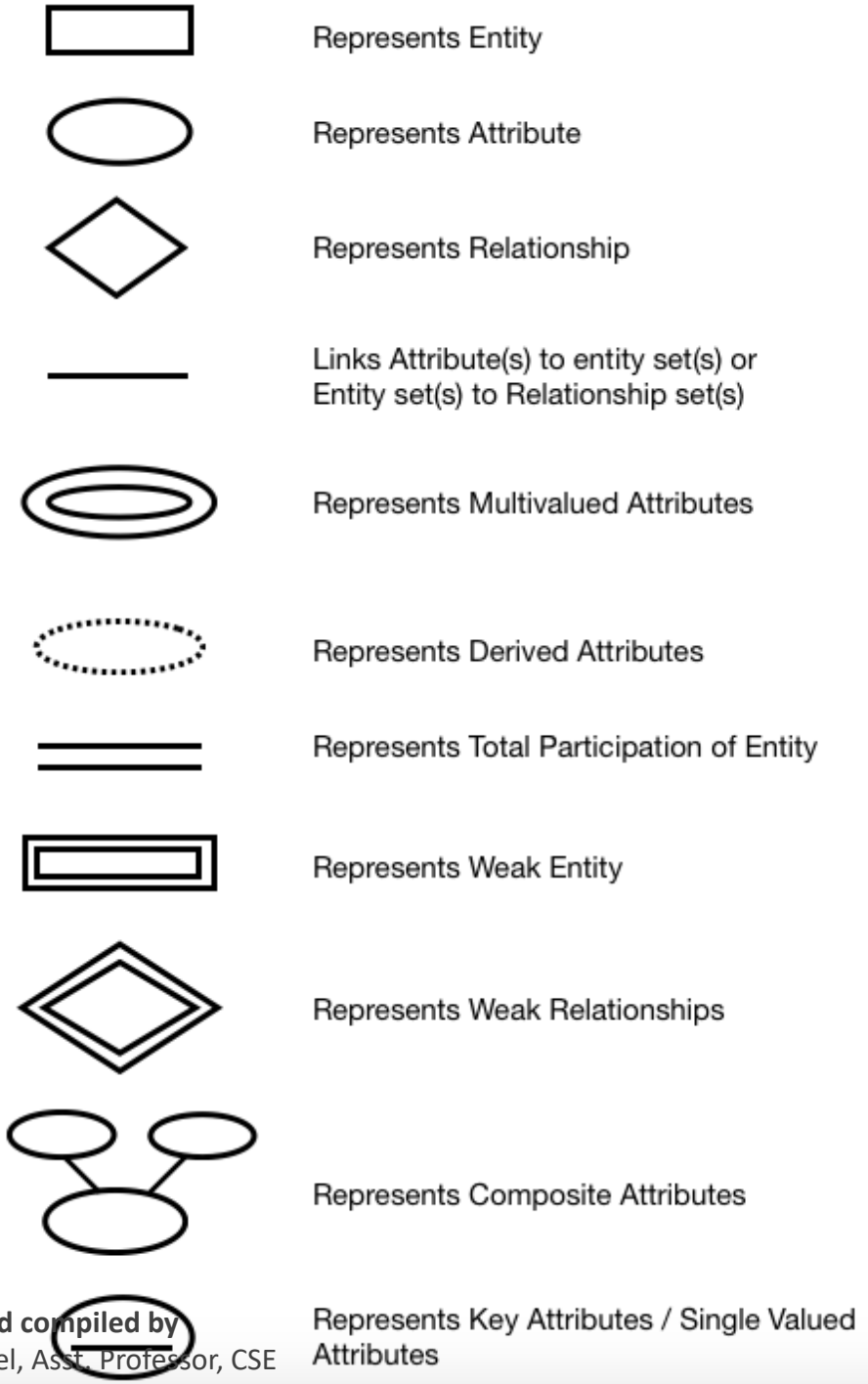
ER (Entity Relationship) Models :

- ER Model is a high-level conceptual data model diagram.
- ER modeling helps you to analyze data requirements to produce a well-designed database.
- The Entity-Relation model represents real-world entities and the relationship between them.
- In other words, ER diagrams help to explain the logical structure of databases.
- **Components of ER Model:**
  - **Entity**
  - **Relationship**
  - **Attributes**

Components in ER Model:



Symbols Used in ER:



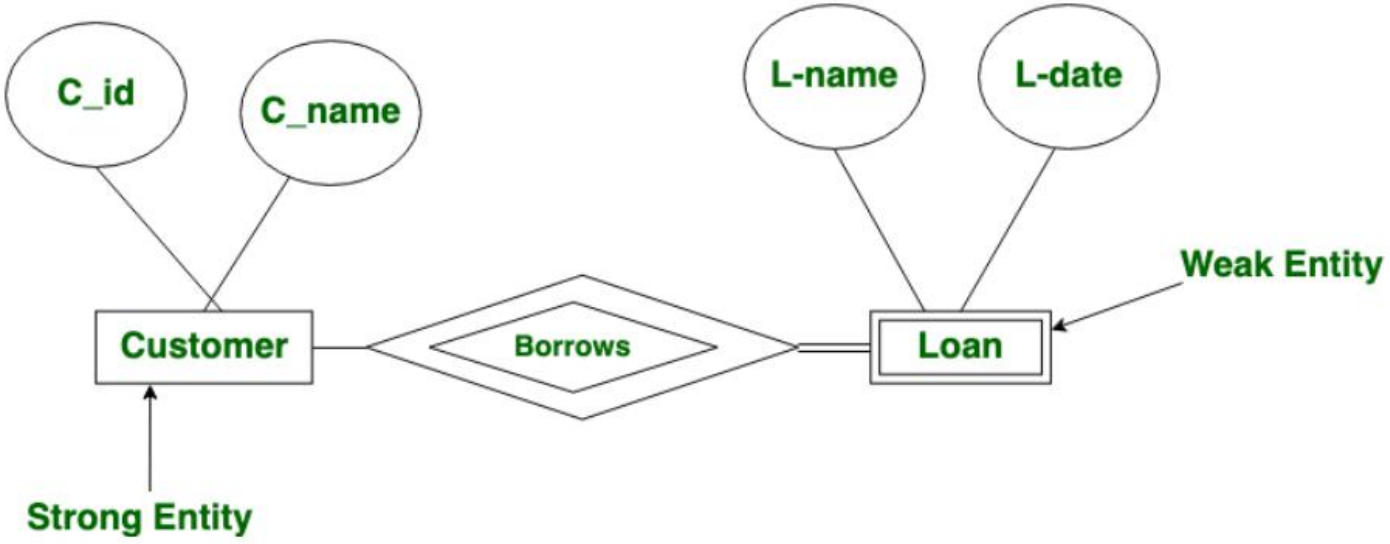
Entity & Entity Types:

- An Entity may be an object with a physical existence – a person, car, house or employee.
- It may be an object with a conceptual existence – a company, a job, or a university course.
- An Entity is an object of Entity Type and set of all entities is called as entity set.
- An entity is represented as rectangle in an ER diagram.
- For example: In the following ER diagram we have two entities Student and College-



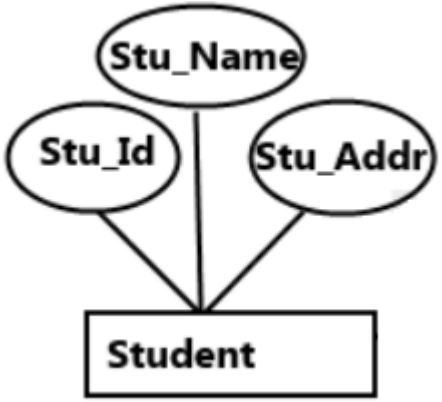
Types of Entities:

- **Strong Entity:** An entity that contains a unique key is called strong entity.
- **Weak Entity Sets:** The entity set which does not have sufficient attributes to form a primary key is called as Weak entity set.
- Weak entity is represented by a double rectangle.



Attributes & Types:

- An attribute describes the property of an entity.
- An attribute is represented as Oval in an ER diagram.

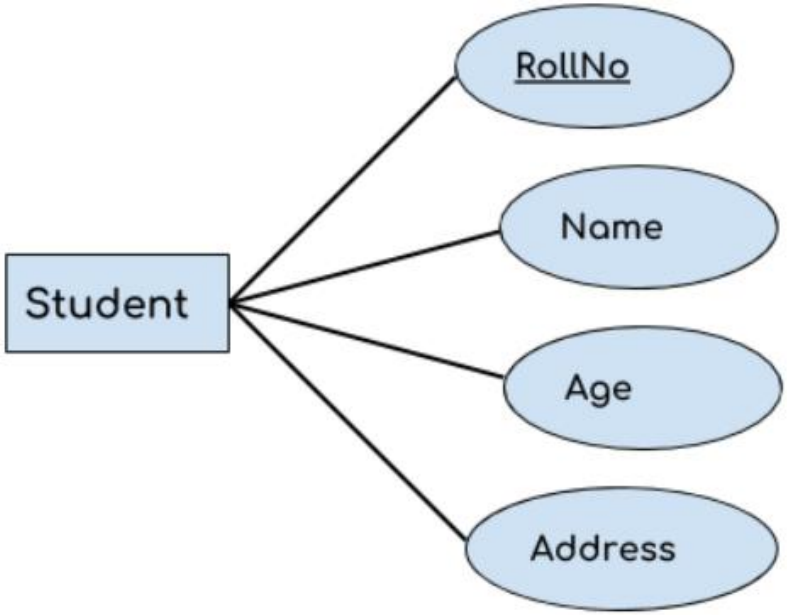


- **There are four types of attributes:**

1. Key attribute
2. Simple & Composite attribute
3. Single valued & Multivalued attribute
4. Stored & Derived attribute

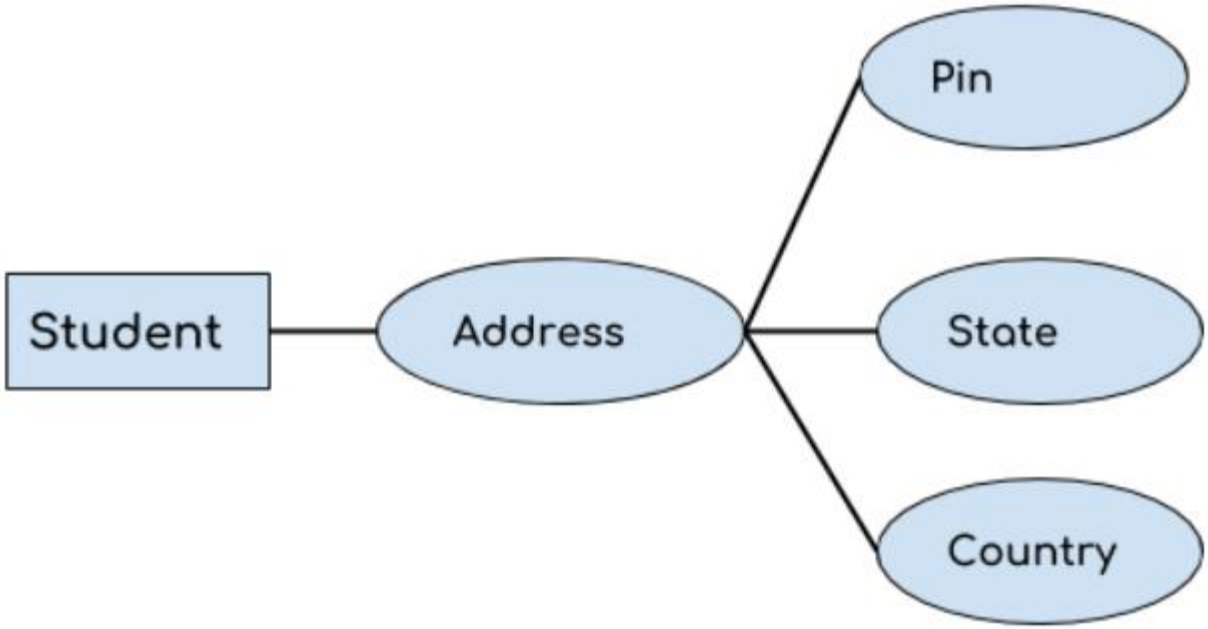
1. Key Attribute:

- A key attribute can uniquely identify an entity from an entity set.
- Key attribute is represented by oval same as other attributes however the text of key attribute is underlined.



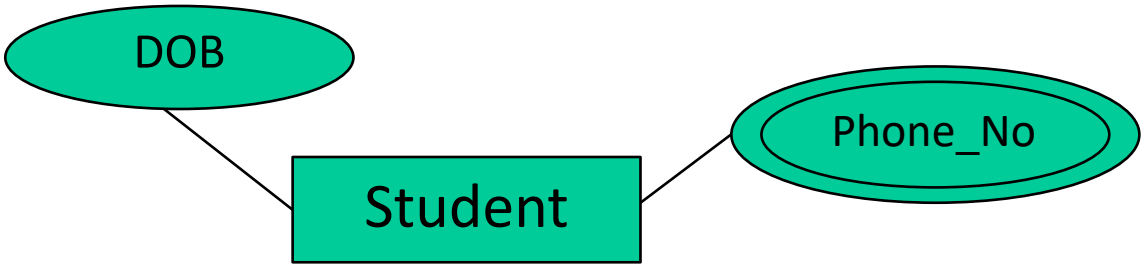
2. Simple & Composite Attributes:

– An attribute that is a combination of other attributes is known as composite attribute.



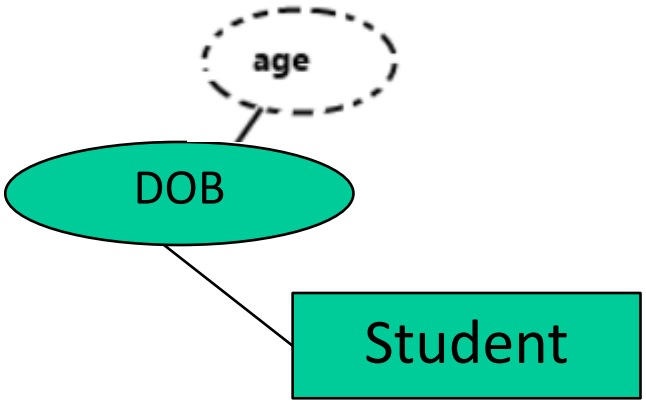
3. Single Valued & Multivalued Attributes:

- An attribute that can hold multiple values is known as multivalued attribute.
- It is represented with **double ovals** in an ER Diagram.
- For example – A person can have more than one phone numbers so the phone number attribute is multivalued.

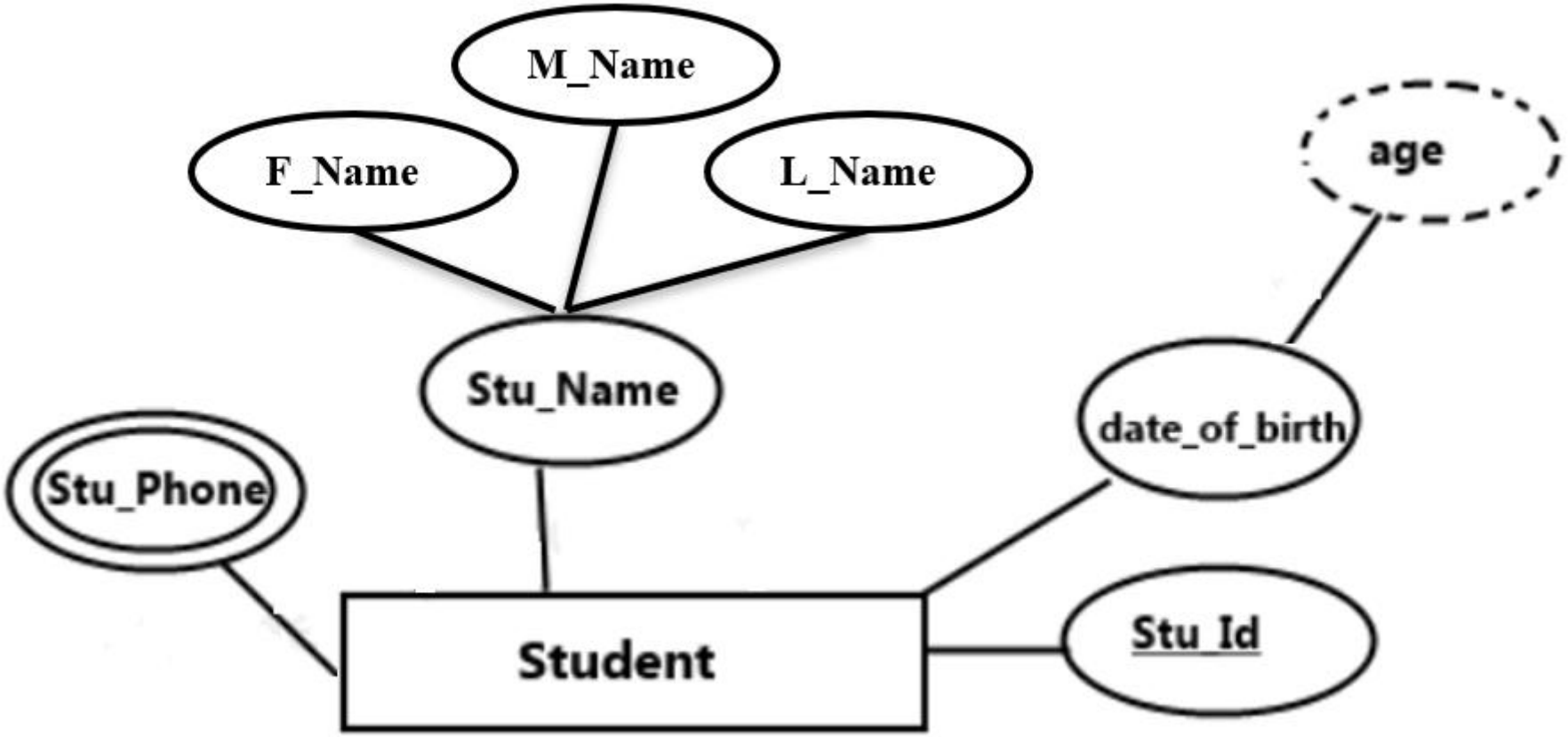


4. Stored & Derived Attributes:

- A derived attribute is one whose value is dynamic and derived from another attribute.
- It is represented by dashed oval in an ER Diagram.



Types of Attributes:



Relationship in ER Model:

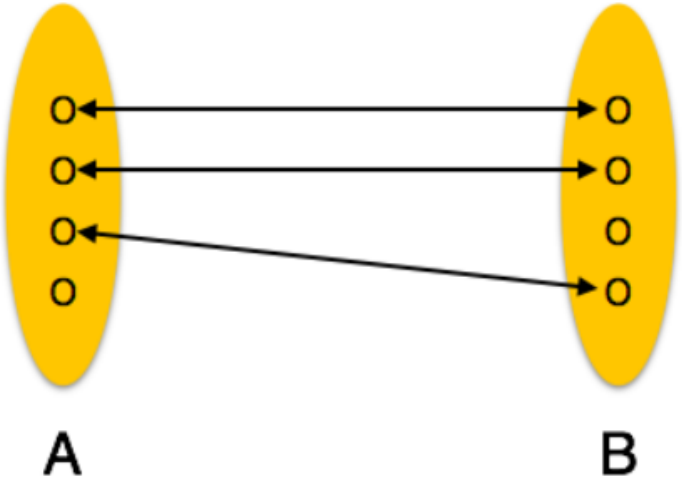
- Association among the entities is called relationship.
- A relationship is represented by diamond shape in ER diagram, it shows the relationship among entities.

- There are four types of relationships:

  1. One to One
  2. One to Many
  3. Many to One
  4. Many to Many.

1. One to One Relationship:

- When a single instance of an entity is associated with a single instance of another entity then it is called one to one relationship.

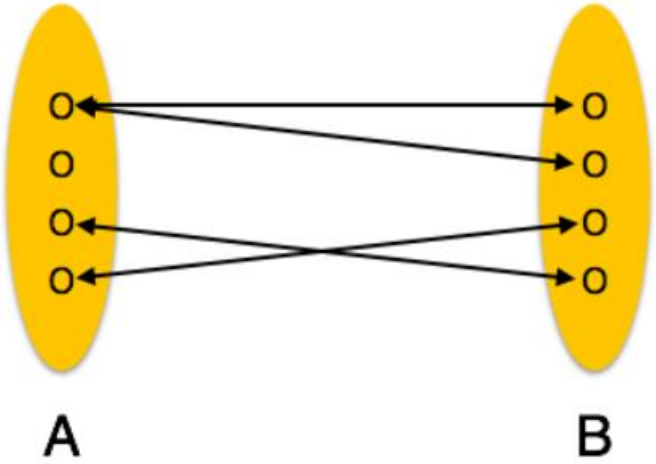


- For example, a person has only one passport and a passport is given to one person.



2. One to Many Relationship:

- When a single instance of an entity is associated with more than one instances of another entity then it is called one to many relationship.

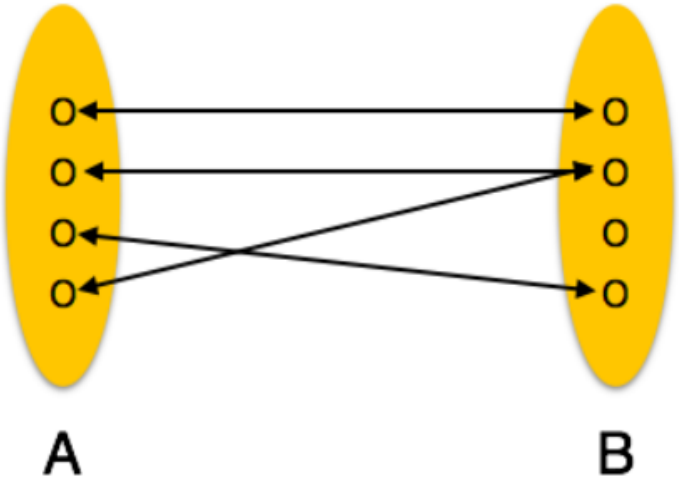


- For example – a customer can place many orders but a order cannot be placed by many customers.



3. Many to One Relationship:

- When more than one instances of an entity is associated with a single instance of another entity then it is called many to one relationship.

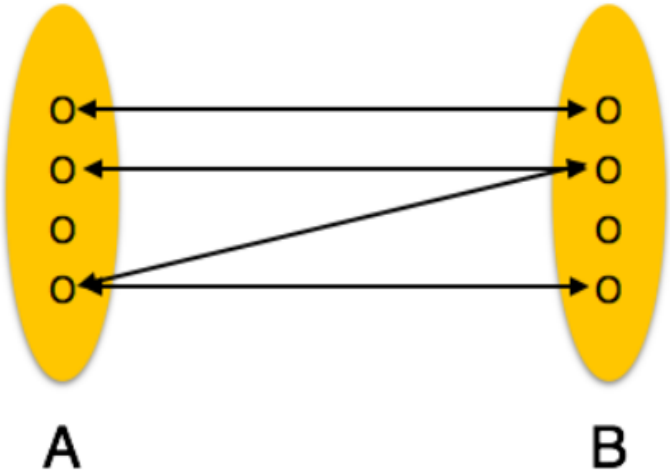


- For example – many students can study in a single college but a student cannot study in many colleges at the same time.



4. Many to Many Relationship:

- When more than one instances of an entity is associated with more than one instances of another entity then it is called many to many relationship.

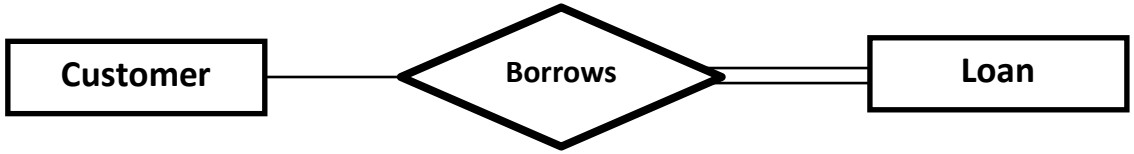


- For example, a student can be assigned to many projects and a project can be assigned to many students.

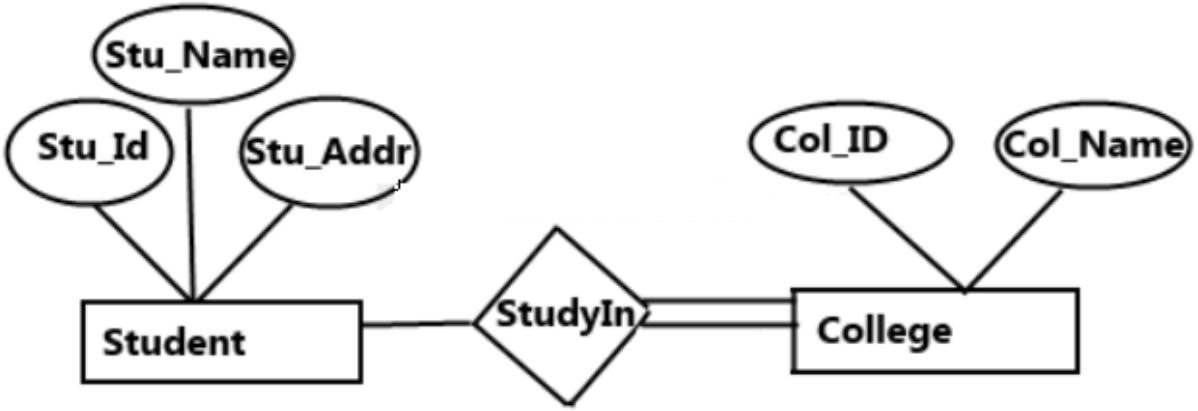


Participation Constraints:

- **Total Participation** – Each entity in the entity set occurs at least one relationship in that relationship set. Total participation is represented by double lines.



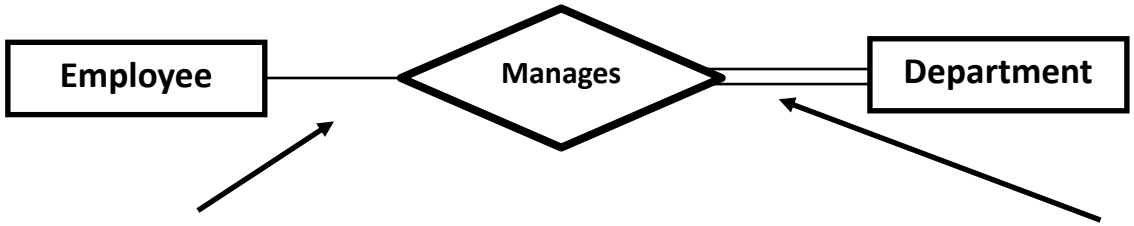
Here each loan entity is associated with at least one associated customer.



E-R Digram with total participation of College entity set in StudyIn relationship Set - This indicates that each college must have atleast one associated Student.

Participation Constraints:

- **Partial Participation** – Each entity in the entity set may not occur in at least one relationship in that relationship set.



Not every employee manages a department.

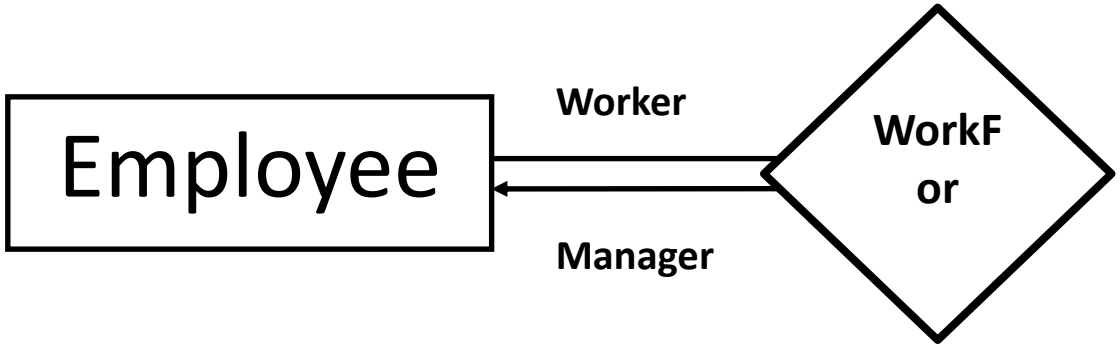
So here is partial participation of employee in 'manages' relationship

Every department is managed by at least one employee (called Manager).

So here is total participation of department in manages relationship

Recursive Relationship:

- If the same entity participates more than once in a relationship it is known as a recursive relationship.
- In the below example an employee can be a manager and can be worker, so there is a recursive relationship.



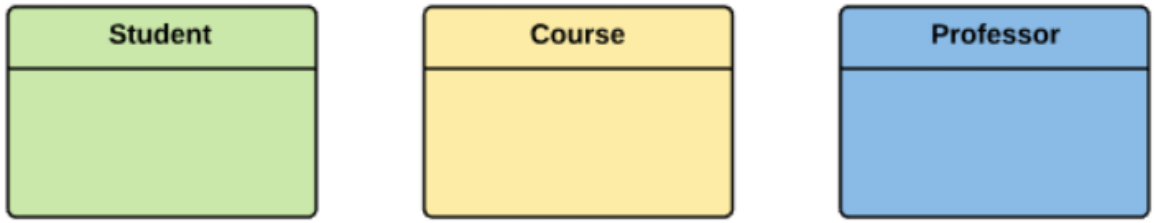
Steps to create an ER Diagram:



In a university, a Student enrolls in Courses. A student must be assigned to at least one or more Courses. Each course is taught by a single Professor. To maintain instruction quality, a Professor can deliver only one course.

Step 1) Entity Identification

- **Student**
- **Course**
- **Professor**

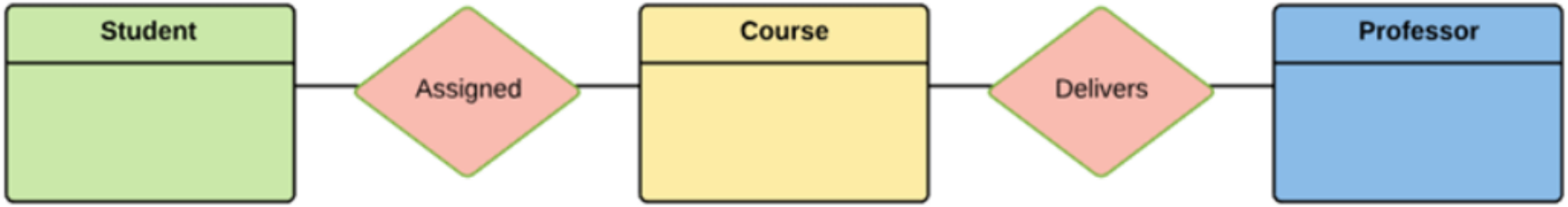


Steps to create an ER Diagram:

Step 2) Relationship Identification

We have the following two relationships

- The student is **assigned** a course
- Professor **delivers** a course

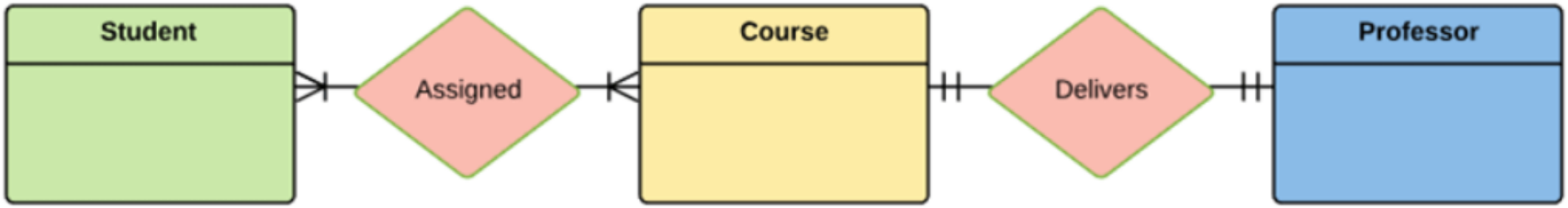


Steps to create an ER Diagram:

Step 3) Cardinality Identification

For them problem statement we know that,

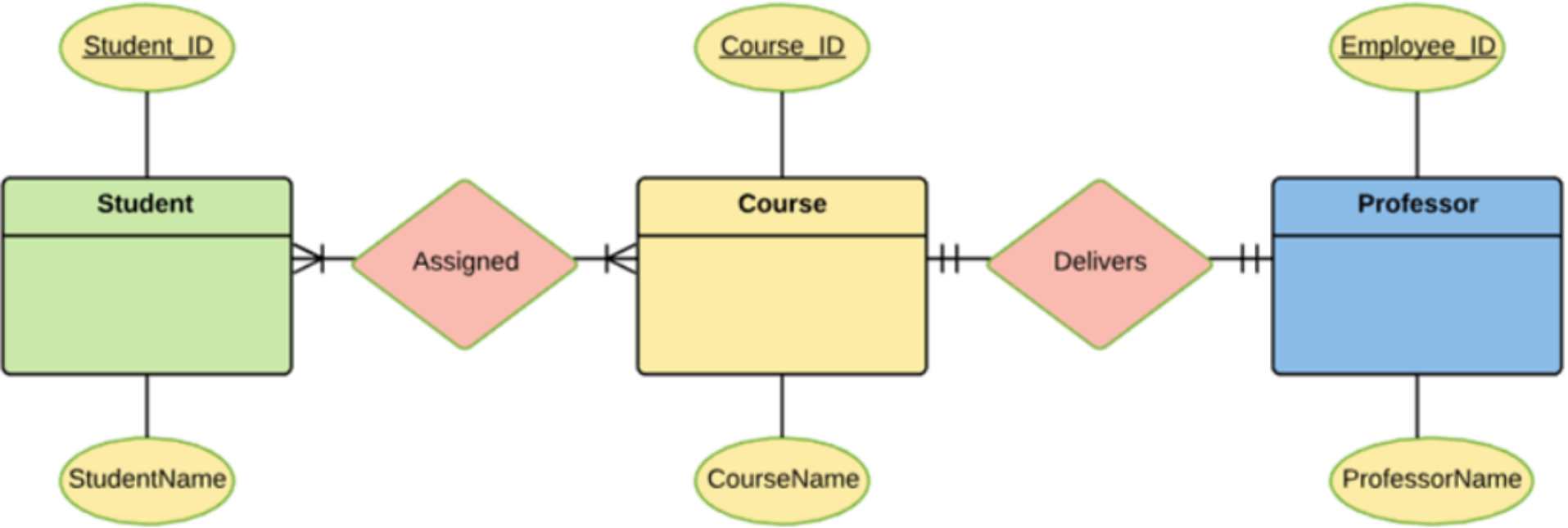
- Many students can be assigned to **multiple** courses
- A Professor can deliver only **one** course



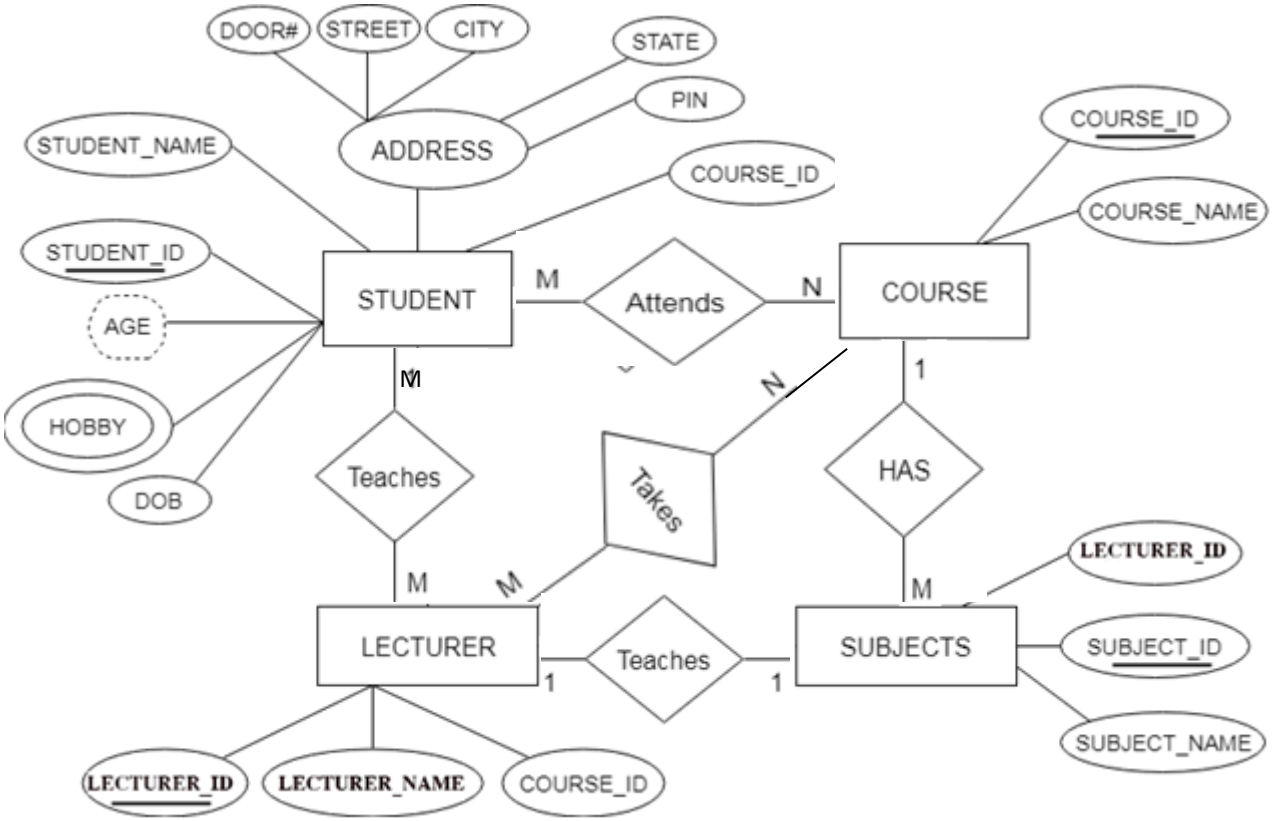
Steps to create an ER Diagram:

Step 4) Identify Attributes

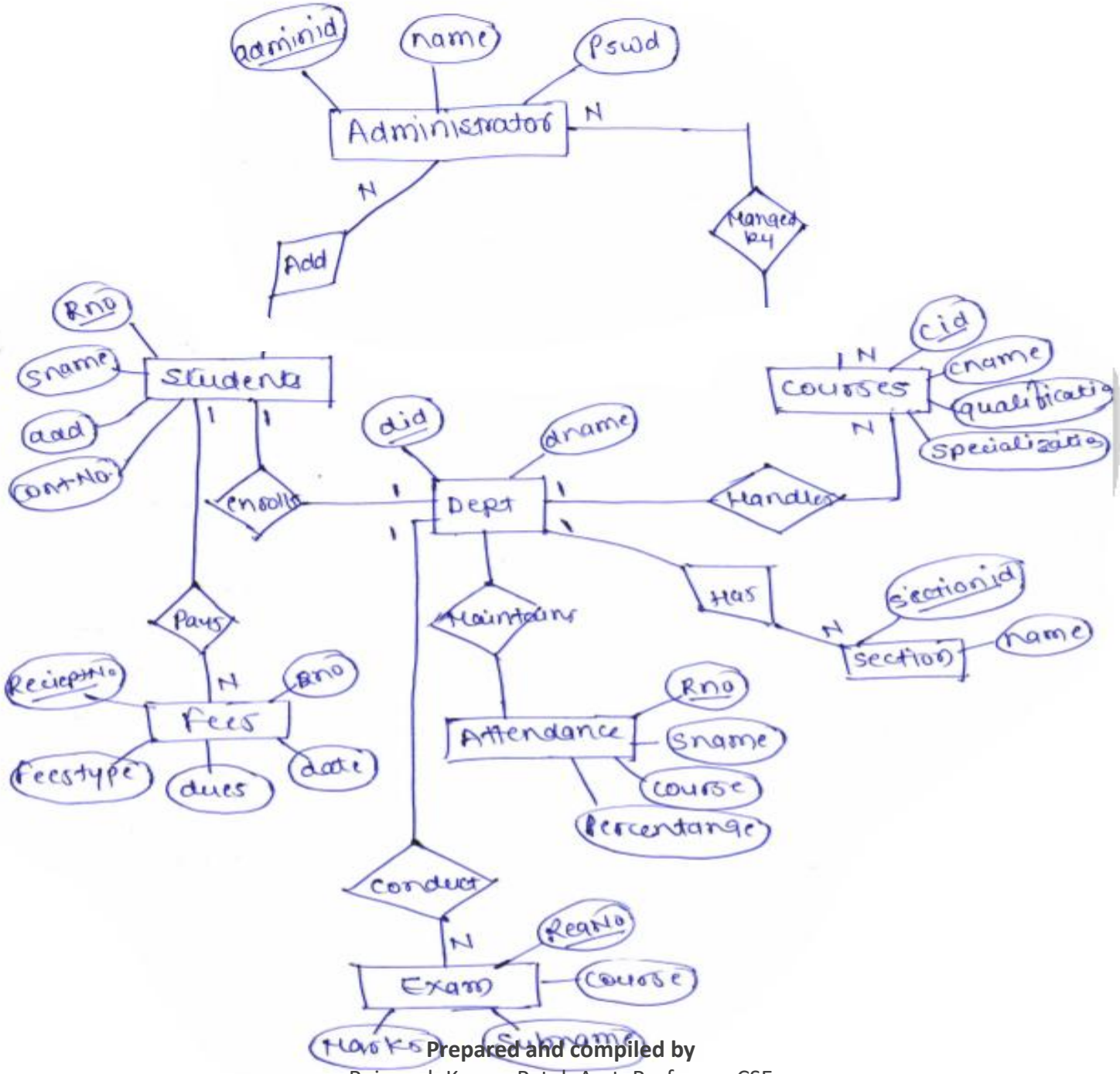
- Initially, it's important to identify the attributes without mapping them to a particular entity.
- Ensure an attribute is to be paired with exactly one entity.



Student Class Management ER Diagram:



Prepared and compiled by
Rajneesh Kumar Patel, Asst. Professor, CSE

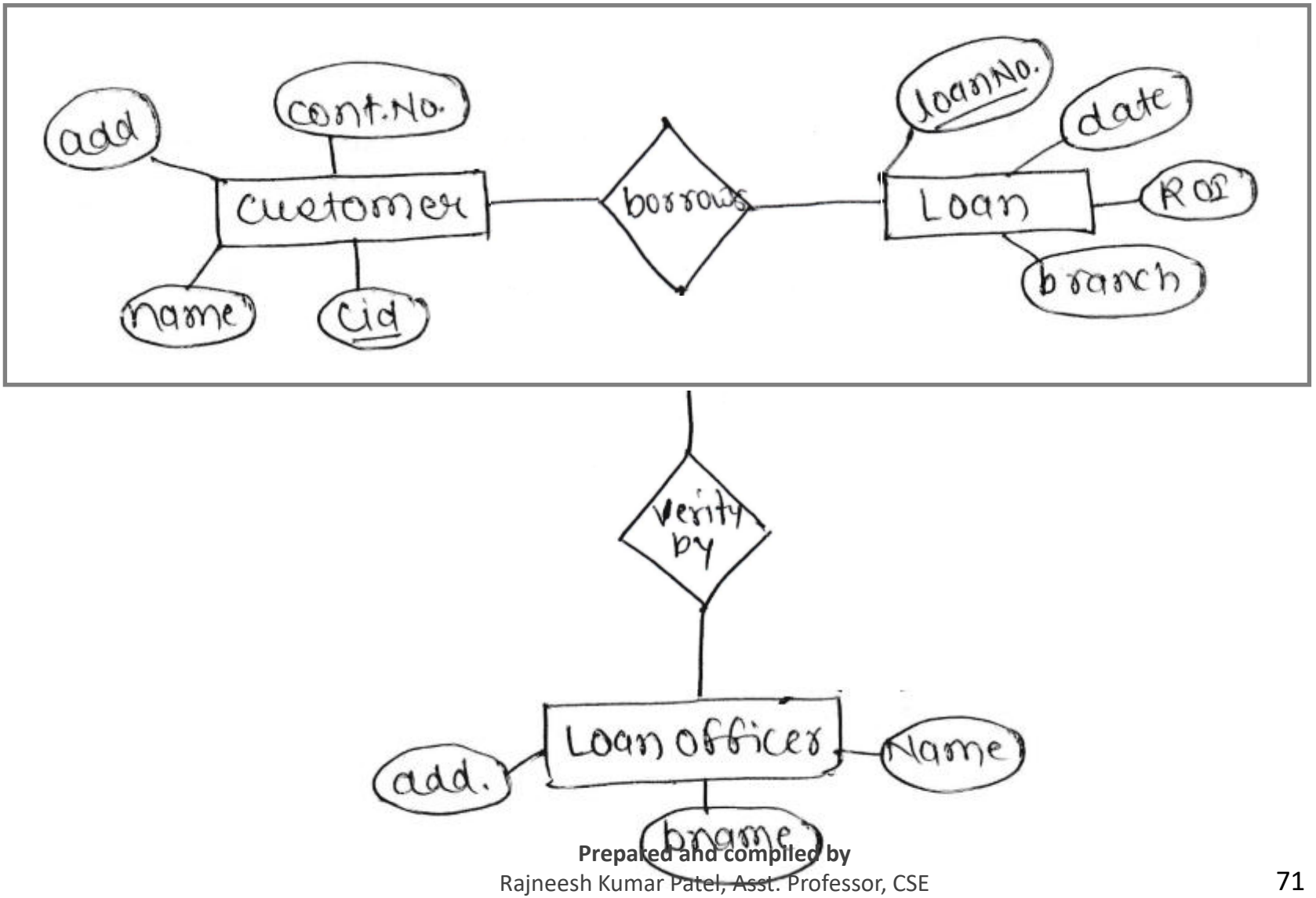

adminid
name
Pswd
Administrator
N
N
Add
Manged by
Rno
Sname
add
CONT-NO
Students
1
1
enrolls
did
dname
Dept
1
1
1
1
Handles
N
1 N
Cid
cname
COURSES
qualificati
Specializatio
Has
N
Sectionid
name
Section
Pays
N
Maintains
Receipt No
Rno
Fees
Feestype
dues
date
Rno
Attendance
Sname
course
Percentange
conduct
N
RegNo
Exam
course
Marks
Subname

Extended Features in ER Diagram:

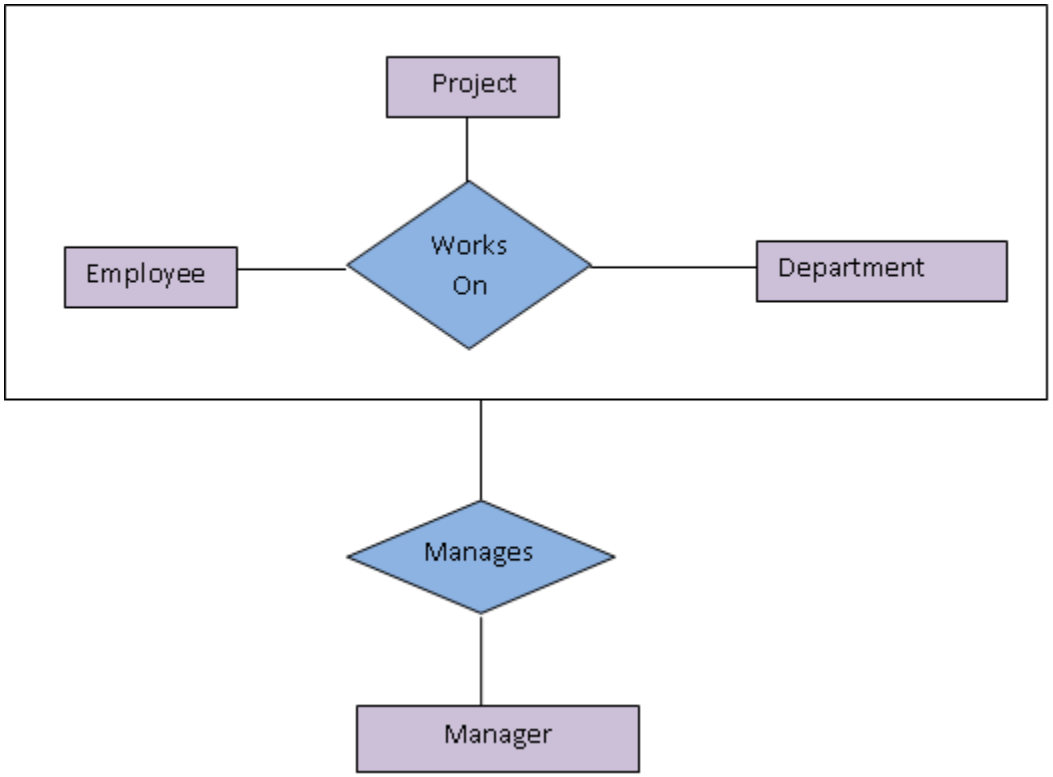
Aggregation:

- Relationship among the relationship.
- It is used to represent a relationship between a whole object and its component part.
- It allows us to indicate that a relationship set participates in another relationship set.
- It is an abstraction for building composite objects from their component objects.
- It is represented by dashed line in an ER diagram.

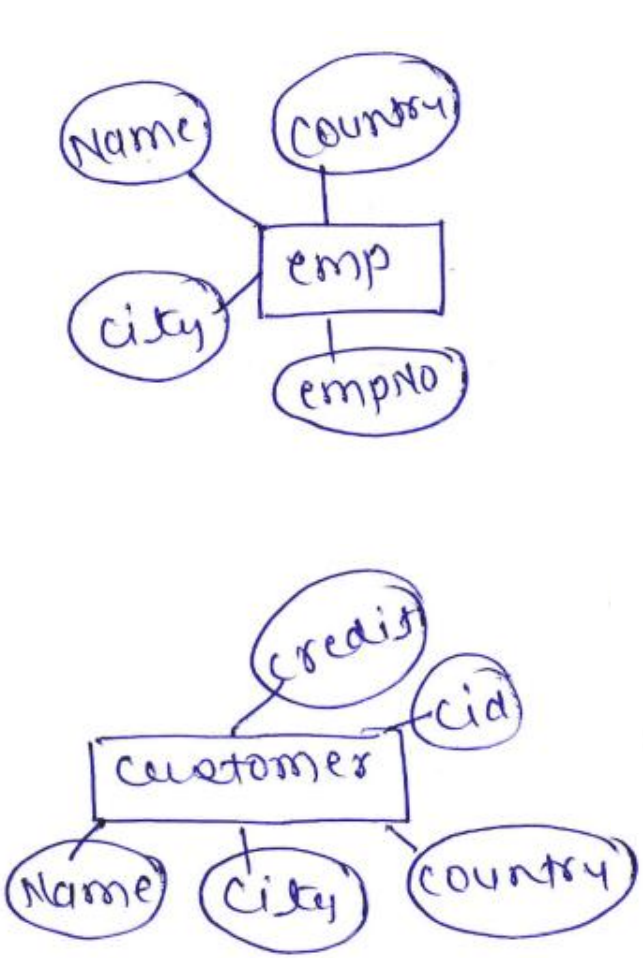
Aggregation in ER Diagram:

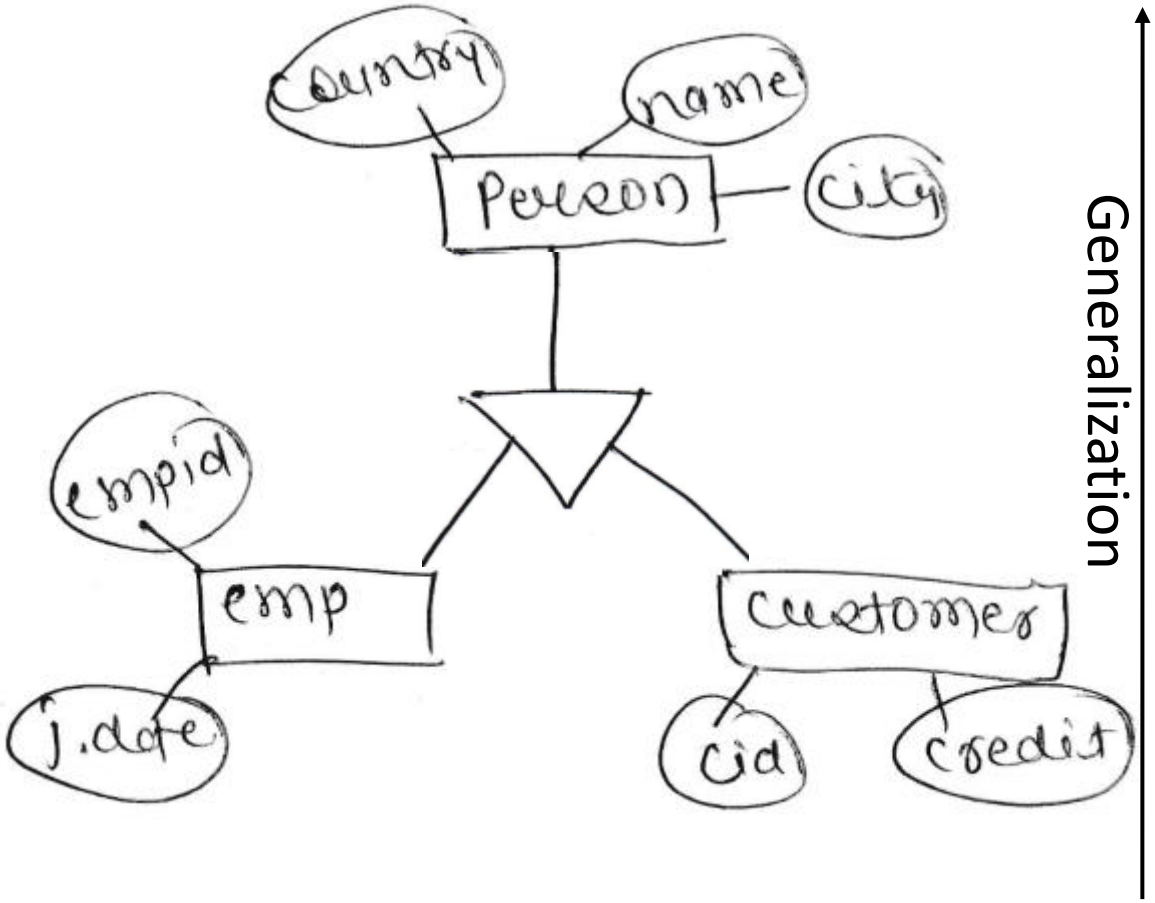


Aggregation in ER Diagram:

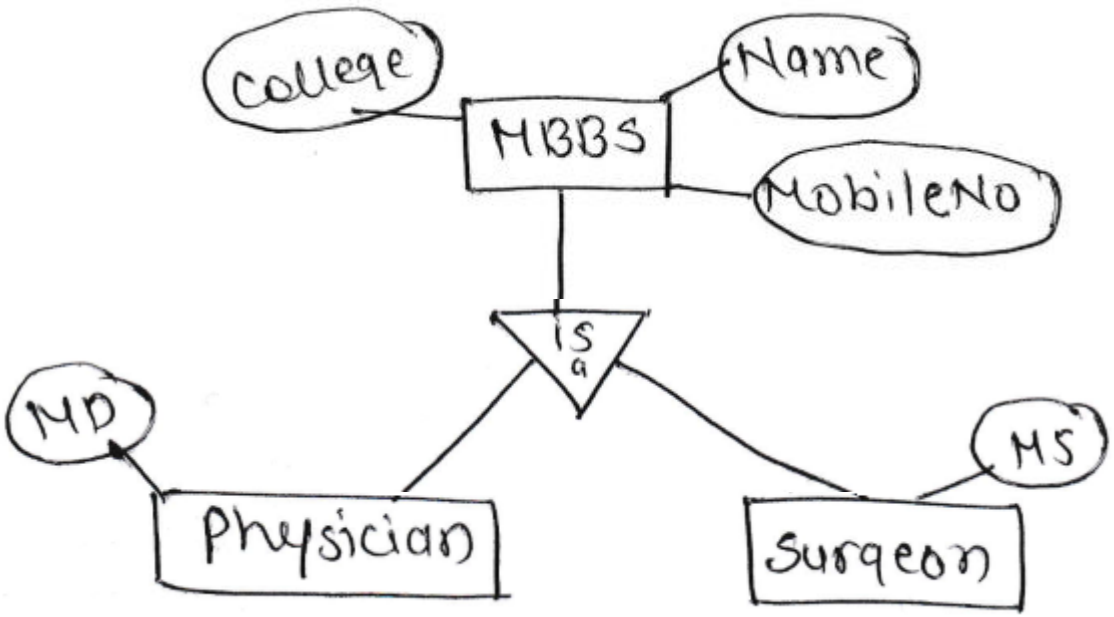


Generalization in ER Diagram:



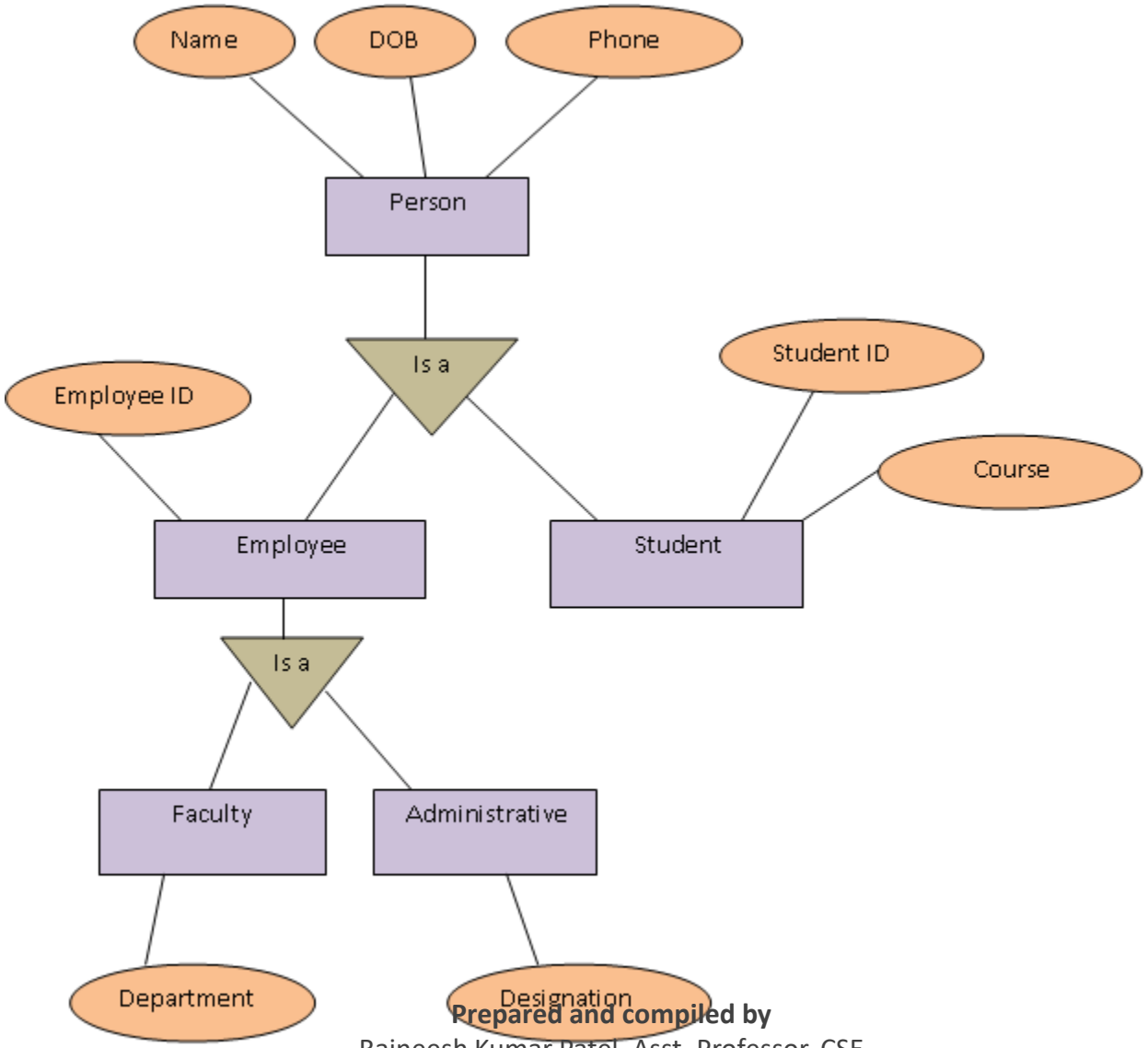


Specialization in ER Diagram:

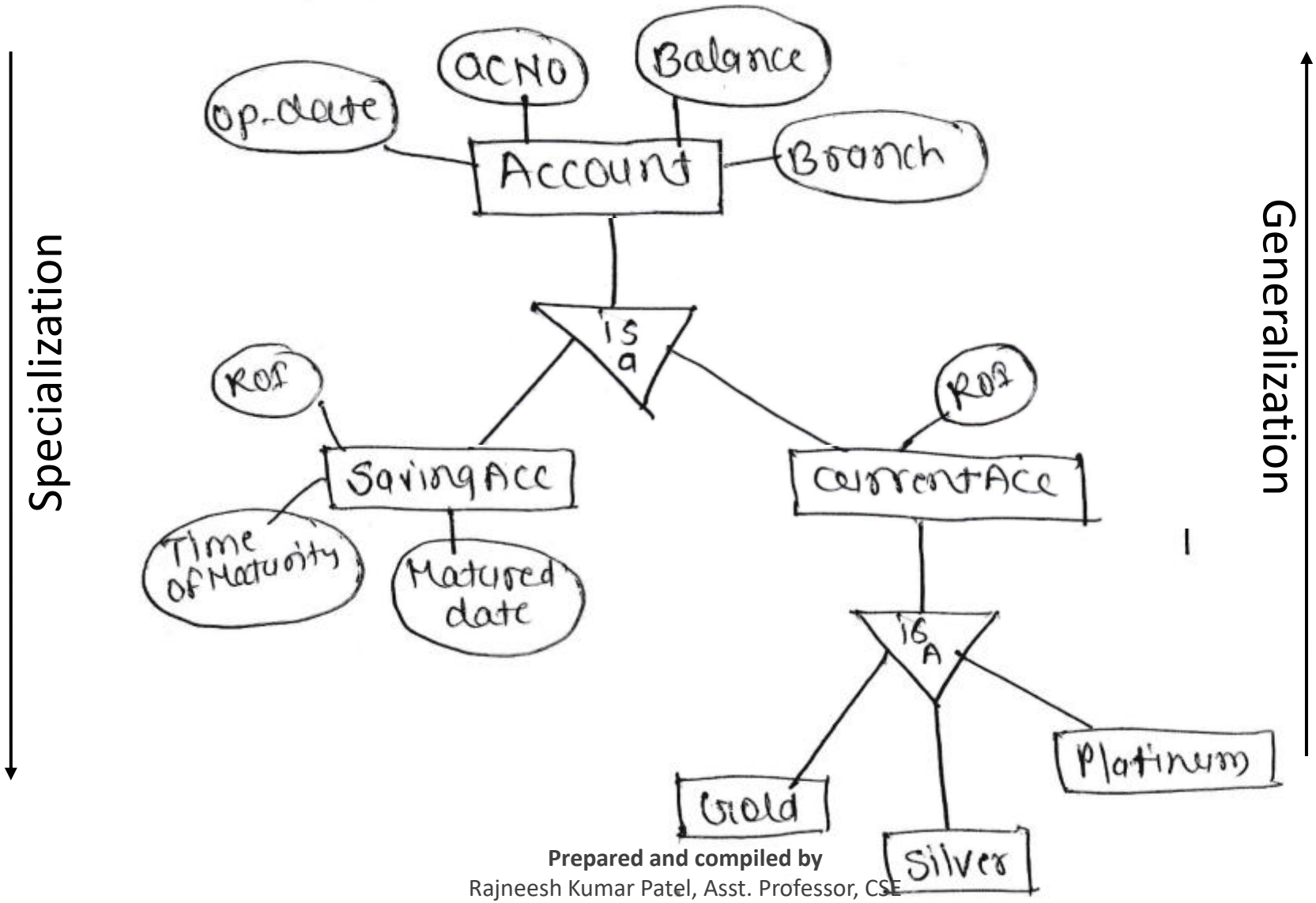


Specialization

Specialization & Generalization in ER Diagram:

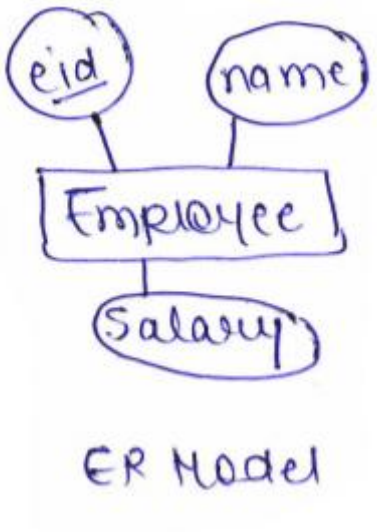


Specialization & Generalization in ER Diagram:



Conversion of ER Model into Relational Model

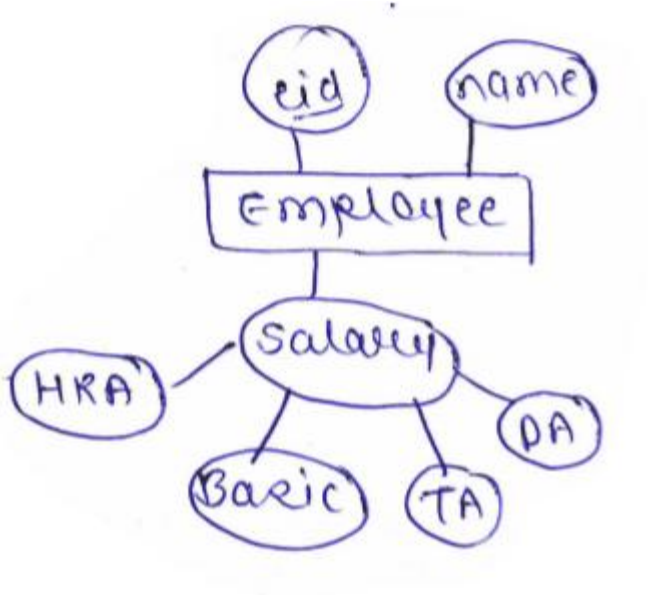
1. ER with Key Attribute:



Employee		
Eid	**Name**	**Salary**
1201	Aman	56,000
1202	Prateek	49,000
1203	Bhavya	52,000

Conversion of ER Model into Relational Model

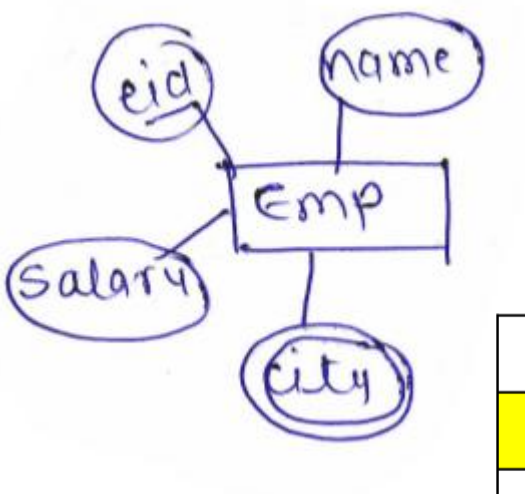
2. Entity set with Composite Attribute:



Employee					
Eid	**Name**	**Basic**	**HRA**	**TA**	**DA**
1201	Aman	32000	3000	1500	4500
1202	Prateek	29000	3000	1500	4500
1203	Bhavya	30000	3000	1500	4500

Conversion of ER Model into Relational Model

3. ER with Multivalued Attribute:



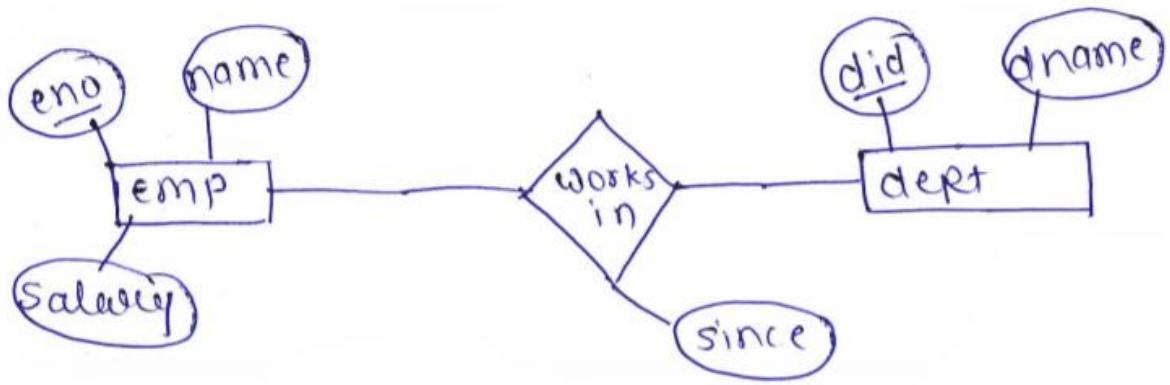
Employee			
Eid	**Name**	**Salary**	**City**
1201	Aman	56,000	Indore, Bhopal
1202	Prateek	49,000	Indore
1203	Bhavya	52,000	Bhopal

Employee		
Eid	**Name**	**Salary**
1201	Aman	56,000
1202	Prateek	49,000
1203	Bhavya	52,000

Employee	
Eid	**City**
1201	Indore
1201	Bhopal
1202	Indore
1203	Bhopal

Conversion of ER Model into Relational Model

4. Translating Relationship set into Relations:



Employee		
Eid	**Name**	**Salary**
1201	Aman	56,000
1202	Prateek	49,000
1203	Bhavya	52,000
1204	Ravish	53,000

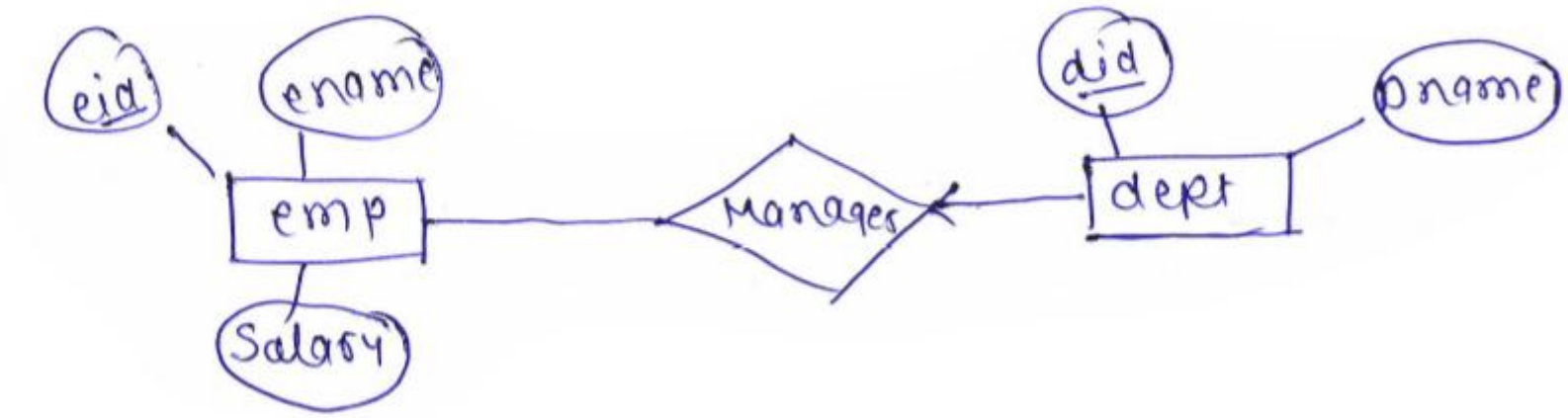
Works_in		
Eid	**Did**	**Since**
1201	D02	01-02-2018
1202	D01	18-05-2020
1202	D02	18-05-2020
1203	D03	01-07-2020
1204	D02	16-01-2019
1204	D03	16-01-2019

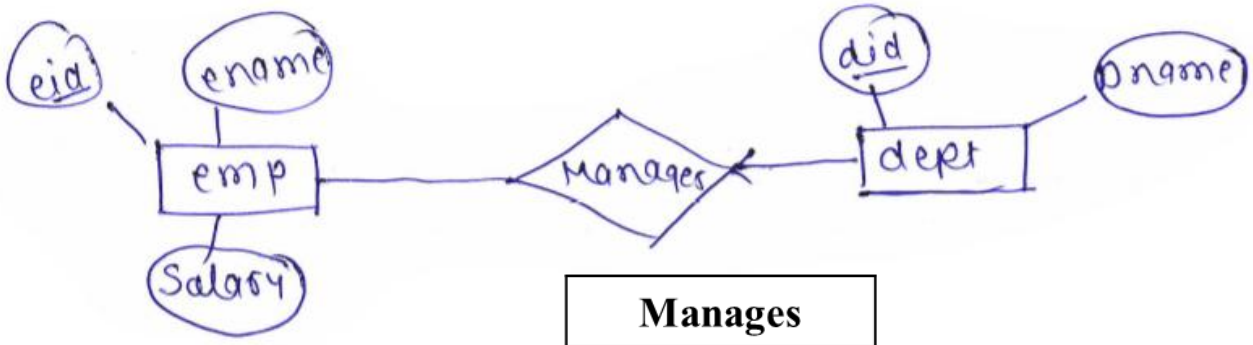
Department	
Did	**Dname**
D01	HR
D02	Admin
D03	Finance

Conversion of ER Model into Relational Model

5. Relationship with key constraints: (One to Many)

Each department is required to have **at most** one employee as manager





Employee		
Eid	**Name**	**Salary**
1201	Aman	56,000
1202	Prateek	49,000
1203	Bhavya	52,000
1204	Ravish	53,000

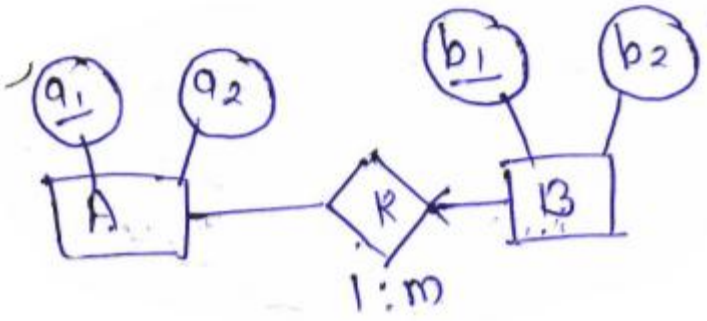
Manages	
Eid	**Did**
1201	D02
1202	D01
1202	D02
1203	D03
1204	D02
1204	D03

Department	
Did	**Dname**
D01	HR
D02	Admin
D03	Finance

Department		
Did	**Dname**	**Eid**
D01	HR	1202
D02	Admin	1202
D03	Finance	1203

Conversion of ER Model into Relational Model

6.1 One to Many:



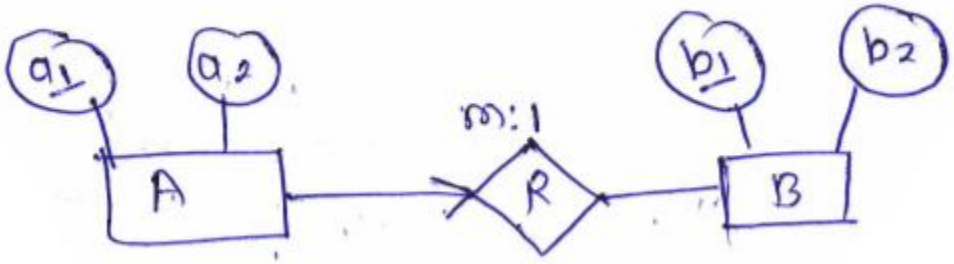
Then No. of Tables will be →

one for A ($\underline{a_1}$, a_2)

and one for B ($\underline{b_1}$, b_2, a_1) ← Foreign key.

Conversion of ER Model into Relational Model

6.2 Many to One:



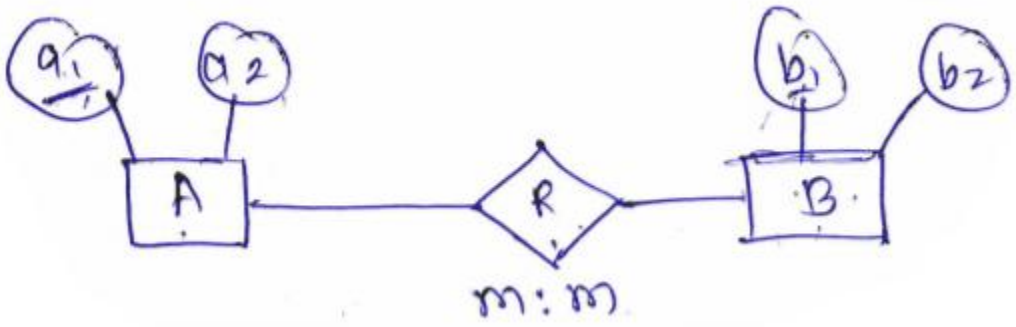
⇒ 2 tables Required

A ($\underline{a_1}$, a_2, b_1) B ($\underline{b_1}$, b_2)

F.K.

Conversion of ER Model into Relational Model

6.3 Many to Many:



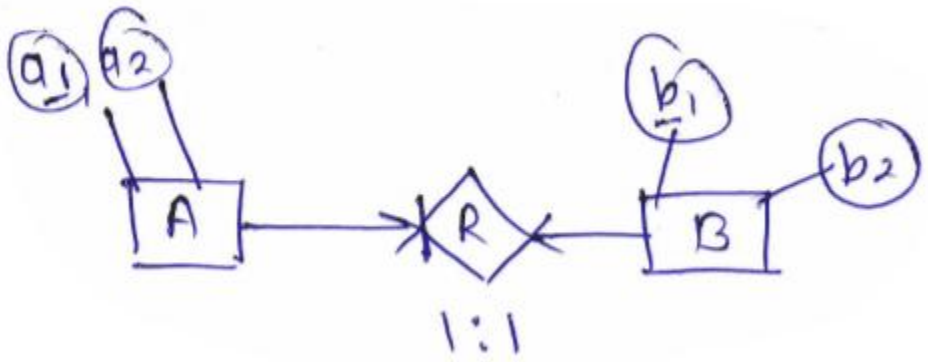
Since there is no key constraint so
No. of tables will be - (3)

one for A ($\underline{a_1}$, a_2)
one for B ($\underline{b_1}$, b_2)
one for R ($\underline{a_1}$, $\underline{b_1}$)

FK

Conversion of ER Model into Relational Model

6.4 One to One:



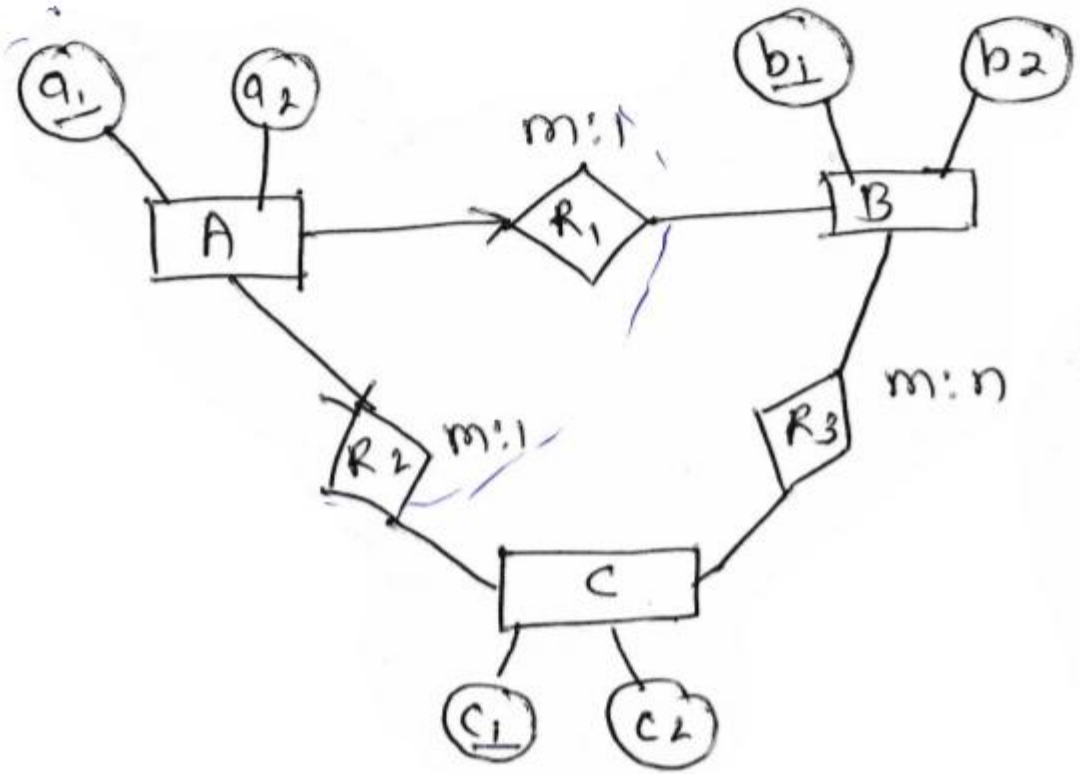
Max. no. of tables = (2)

either A ($\underline{a_1}$, a_2, b_1) B ($\underline{b_1}$, b_2)

or A ($\underline{a_1}$, a_2) B ($\underline{b_1}$, b_2, a_1)

Conversion of ER Model into Relational Model

Find the minimum no. of tables that are possible for converting into ER Model



require 4 tables. → F.K

AR1R2 (a1, a2, c1, b1)

B (b1, b2)

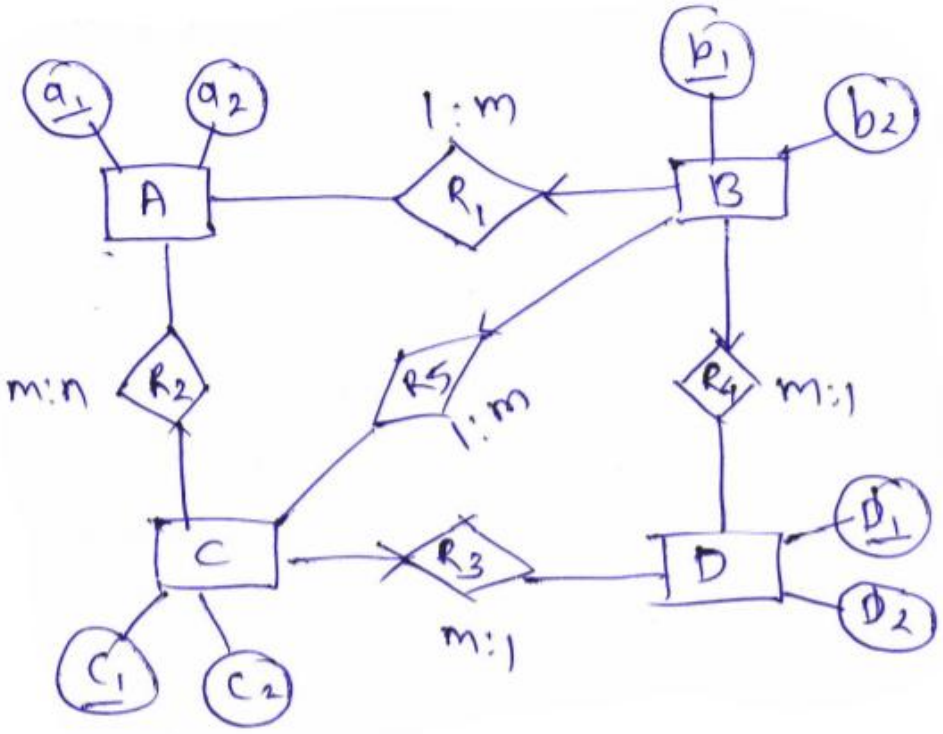
C (c1, c2)

R3 (b1, c1)

F.K.

Conversion of ER Model into Relational Model

Find the minimum no. of tables that are possible for converting into ER Model



No. of tables = 5

(i) A (a1, a2)

(ii) D (d1, d2)

(iii) BR1R4R5 (b1, b2, d1, c1, a1)

F.K.

(iv) R2 (a1, c1)

(v) CR3 (c1, c2, d1)

Thank You